

Методы гибкой разработки программного обеспечения: Обзор и анализ

Авторы: Пекка Абрахамссон, Оути Сало, Юсси Ронкайнен и Юхани.
Варста

Это авторская версия работы. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ronkainen, J. &

Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, публикация VTT 478, Эспоо, Финляндия, 107 стр.

Версию правообладателя можно загрузить по адресу <http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Это авторская версия работы. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ronkainen, J. &

Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, публикация VTT 478, Эспоо, Финляндия, 107 стр.

Версию правообладателя можно загрузить по адресу <http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Ключевые слова: Разработка программного обеспечения, управление программными проектами, гибкий процесс, легковесный
процесс, экстремальное программирование, разработка на основе функций, метод разработки
динамических систем, Scrum, прагматичное программирование, гибкое моделирование, разработка
программного обеспечения с открытым исходным кодом, рациональный унифицированный процесс,
адаптивная разработка программного обеспечения, семейство методологий Crystal

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
 Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
 Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Содержание

1. Введение.....	9
2. Обзор, определения и характеристики Agile.....	11
Введение.....	11
2.2. Обзор и определения.....	13
2.3 Характеристика.....	16
2.4. Резюме.....	19
3. Существующие гибкие методы.....	20
3.1. Экстремальное программирование.....	20
3.1.1. Процесс.....	21
3.1.2 Роли и обязанности.....	23
3.1.3 Практики.....	24
3.1.4. Внедрение и опыт применения.....	27
3.1.5. Область применения.....	28
3.1.6. Текущие исследования.....	29
3.2. Скрам.....	29
3.2.1.Процесс.....	30
3.2.2 Роли и обязанности.....	32
3.2.3 Практики.....	33
3.2.4. Внедрение и опыт применения.....	36
3.2.5. Область применения.....	37
3.2.6. Текущие исследования.....	37
3.3 Семейство методологий Crystal.....	38
3.3.1 Процесс.....	40
3.3.2 Роли и обязанности.....	44
3.3.3 Практики.....	45
3.3.4. Внедрение и опыт применения.....	47
3.3.5. Область применения.....	48
3.3.6. Текущие исследования.....	48
3.4. Разработка на основе функций.....	49
3.4.1. Процесс.....	49
3.4.2 Роли и обязанности.....	52
3.4.3 Практики.....	55

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
 Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
 Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.4.4. Внедрение и опыт применения.....	56	3.4.5.
Область применения.....	56	3.4.6. Текущие
исследования.....	57	
3.5. Рациональный унифицированный процесс.....	57	
3.5.1.Процесс.....	57	
3.5.2 Роли и обязанности.....	60	3.5.3
Практики.....	61	
3.5.4. Внедрение и опыт применения.....	62	3.5.5. Область
применения.....	63	3.5.6. Текущие
исследования.....	63	
3.6 Метод разработки динамических систем.....	63	3.6.1
Процесс.....	64	
3.6.2 Роли и обязанности.....	67	3.6.3
Практики.....	68	
3.6.4. Внедрение и опыт применения.....	70	3.6.5. Область
применения.....	71	3.6.6. Текущие
исследования.....	72	
3.7 Разработка адаптивного программного обеспечения.....	72	3.7.1
Процесс.....	72	
3.7.2 Роли и обязанности.....	76	3.7.3
Практики.....	76	
3.7.4. Внедрение и опыт.....	76	3.7.5. Текущие
исследования.....	77	
3.8 Разработка программного обеспечения с открытым исходным кодом		
78 3.8.1 Процесс	79	
3.8.2 Роли и обязанности.....	80	3.8.3
Практики.....	82	
3.8.4. Внедрение и опыт применения.....	83	3.8.5.
Область применения.....	84	3.8.6. Текущие
исследования.....	84	
3.9 Другие гибкие методы.....	86	3.9.1 Гибкое
моделирование.....	86	3.9.2 Прагматичное
программирование.....	88	
4. Сравнение гибких методов.....	91	4.1.
Введение.....	91	
4.2 Общие характеристики.....	92	

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

4.3. Усыновление..... 97

5. Выводы..... 105

Ссылки..... 108

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

1. Введение

Область разработки программного обеспечения не стесняется внедрять новые методологии. Действительно, за последние 25 лет было предложено множество различных подходов к разработке программного обеспечения, из которых лишь немногие сохранились до наших дней. В недавнем исследовании (Nandhakumar and Avison, 1999) утверждается, что традиционные методологии разработки информационных систем¹ (ИС) «рассматриваются в первую очередь как необходимая фикция, создающая видимость контроля или придающая символический статус». В том же исследовании утверждается, что эти методологии слишком механистичны для детального использования. Парнас и Клементс (1986) ранее выдвигали аналогичные аргументы. Труэкс и др. (2000) занимают крайнюю позицию и утверждают, что традиционные методы, возможно, являются «всего лишь недостижимыми идеалами и гипотетическими „соломенными чучелами“, которые служат нормативным руководством для утопических ситуаций развития». В результате разработки промышленного программного обеспечения стали скептически относиться к «новым» решениям, которые трудно понять и поэтому остаются неиспользуемыми (Wieggers, 1998). На этом фоне зародились гибкие методы разработки программного обеспечения.

Хотя до сих пор не существует единого мнения о том, что именно подразумевается под понятием «agile», оно вызвало большой интерес среди практиков, а в последнее время и в академической среде. Внедрение метода экстремального программирования (более известного как XP, Beck, 1999a; Beck, 1999b) получило широкое признание в качестве отправной точки для различных подходов к гибкой разработке программного обеспечения. С тех пор был также изобретен или заново открыт ряд других методов, которые

По всей видимости, они принадлежат к одному семейству методологий. К таким методам относятся, например, Crystal Methods (Cockburn, 2000), Feature-Driven Development (Palmer and Felsing, 2002) и Adaptive Software Development (Highsmith, 2000). В знак возросшего интереса журнал Cutter IT Journal недавно посвятил три полноценных номера лёгким методологиям, а также

¹ Программная инженерия (ПИ) отличается от области ИС в основном тем, что сообщество ИС учитывает социальные и организационные аспекты (например, Dhillon, 1997; Baskerville, 1998). Более того, ПИ традиционно фокусируется на практических средствах разработки программного обеспечения (Sommerville, 1996). Однако для целей данной публикации такое различие не является необходимым. Таким образом, литература по ИС, касающаяся фактического использования различных методов считаются релевантными.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

участие как минимум в двух крупных международных конференциях пришлось ограничить из-за большого количества участников.

Хотя о фактической отдаче инвестиций в технологические процессы известно мало (Glass, 1999), ещё меньше известно о том, какую выгоду организация получит от использования гибких методологий разработки программного обеспечения. Первые отзывы об опыте применения гибких методологий в отрасли преимущественно положительные (например, Anderson et al., 1998; Williams et al., 2000; Grenning, 2001). Однако на данном этапе получить точные цифры сложно.

Несмотря на высокий интерес к этой теме, до сих пор не достигнуто чёткого согласия относительно того, как отличить гибкую разработку программного обеспечения от более традиционных подходов. Границы, если таковые существуют, чётко не установлены. Однако было показано, что некоторые методы не обязательно подходят всем людям (Naug, 1993) или ситуациям (Baskerville et al.).

1992). По этой причине, например, Хамфри (1995) призывает к разработке индивидуального процесса для каждого разработчика программного обеспечения. Несмотря на эти результаты, мало внимания уделяется анализу того, в каких ситуациях гибкие методы разработки подходят лучше других. Насколько нам известно, систематического обзора методологий гибкой разработки до сих пор не проводилось. В результате в настоящее время отсутствуют процедуры, позволяющие специалистам выбрать метод, наиболее эффективный в данных обстоятельствах.

Поэтому наша цель — начать восполнять этот пробел, систематически изучая существующую литературу по гибким методологиям разработки программного обеспечения. Данная публикация преследует три цели. Во-первых, в результате этого обобщающего анализа предлагается определение и классификация гибких подходов. Во-вторых, дается анализ предлагаемых подходов по заданному критерию, и, в-третьих, сравниваются внедренные методы гибкой разработки с целью выявления их сходств и различий.

Данная работа состоит из пяти разделов. В следующем разделе дано определение метода гибкой разработки программного обеспечения, используемого в контексте данной публикации. В третьем разделе рассматривается большинство существующих методов гибкой разработки программного обеспечения, которые затем сравниваются, обсуждаются и суммируются в четвертом разделе. В шестом разделе публикация завершается заключительными замечаниями.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

2. Обзор, определения и характеристики

Целью данного раздела является характеристика значений, которые в настоящее время ассоциируются с концепцией «agile», и предоставление определения метода гибкой разработки программного обеспечения, используемого в контексте данной публикации.

2.1. Предыстория

Agile, означающий «качество гибкости; готовность к движению; ловкость, активность, сноровка в движении»², — методы разработки программного обеспечения снова пытаются предложить ответ нетерпеливому бизнес-сообществу, требующему более лёгких, но при этом более быстрых и гибких процессов разработки ПО. Это особенно актуально для быстрорастущей и нестабильной индустрии интернет-программного обеспечения, а также для формирующейся среды мобильных приложений. Новые гибкие методы вызвали значительное количество литературы и дискуссий, см., например, следующие статьи в Cutter IT Journal³: The Great Methodologies Debate: Part 1 и Part 2. Однако академические исследования по этой теме всё ещё редки, поскольку большинство существующих публикаций написаны практиками или консультантами.

Труэкс и др. (2000) задаются вопросом, действительно ли разработка информационных систем осуществляется на практике в соответствии с правилами и рекомендациями, установленными многочисленными доступными методами разработки программного обеспечения. Различия между привилегированными и маргинализированными методологическими процессами разработки информационных систем, предложенные авторами, представлены в таблице 1.

Таблица 1. Некоторые различия привилегированных и маргинализированных слоев населения методологические процессы ИСД

² dictionary.oed.com, (2.5.2002)

³ Журнал Cutter IT, ноябрь (т. 14, № 11), декабрь 2001 г. (т. 14, № 12) и Январь 2002 г. (Том 15, № 1)

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Привилегированный методологический текст	Маргинализированный методологический текст
Разработка информационных систем — это:	
Управляемый, контролируемый процесс	Случайный, оппортунистический процесс, вызванный случайностью
Линейный, последовательный процесс	Процессы происходят одновременно, перекрываются и между ними есть пробелы.
Воспроизводимый, универсальный процесс	Встречается в совершенно уникальных и идиографических формах
Рациональный, решительный и целенаправленный процесс	Договорной, компромиссный и капризный

Это сравнение открывает интересную и поучительную перспективу и даёт некоторую основу для анализа гибких методов разработки программного обеспечения (также известных как облегчённые методы). Проекты с привилегированным методом используют общепринятые процессуальные (также известные как основанные на плане) методы разработки программного обеспечения, в то время как маргинализированные методы имеют много общего с новыми гибкими методами разработки программного обеспечения, которые более подробно обсуждаются ниже.

Макколи (2001) утверждает, что основная философия процессно-ориентированных методов заключается в том, что требования к программному проекту полностью фиксируются и замораживаются до начала проектирования и разработки ПО. Поскольку такой подход не всегда осуществим, необходимы также гибкие, адаптивные и динамичные методы, позволяющие разработчикам вносить изменения в спецификации на поздних этапах.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

2.2 Обзор и определения

«Agile-движение» в индустрии программного обеспечения начало свою деятельность с Манифеста гибкой разработки программного обеспечения (Agile Software Development Manifesto)⁴, опубликованного группой специалистов и консультантов в 2001 году (Beck et al., 2001; Cockburn, 2002a). Основные ценности, которым следуют сторонники Agile, представлены ниже:

- Индивидуумы и взаимодействия важнее процессов и инструментов
- Рабочее программное обеспечение с подробной документацией
- Сотрудничество с клиентами в переговорах по контракту
- Реагирование на изменения важнее следования плану

Вот основные ценности, которых придерживается гибкое сообщество:

Во-первых, agile-движение делает акцент на взаимоотношениях и общности разработчиков программного обеспечения, а также на роли человека, отраженной в контрактах, в противовес институционализированным процессам и инструментам разработки. В существующих agile-практиках это проявляется в тесных командных отношениях, тесной организации рабочей среды и других процедурах, укрепляющих командный дух.

Во-вторых, важнейшая задача команды разработчиков — постоянно выпускать протестированное и работающее программное обеспечение. Новые версии выпускаются регулярно, в некоторых случаях даже ежечасно или ежедневно, но чаще всего — раз в два месяца или ежемесячно. Разработчикам настоятельно рекомендуется поддерживать простой, понятный и максимально технически продвинутый код, что позволяет снизить нагрузку на документирование до приемлемого уровня.

В-третьих, взаимоотношениям и сотрудничеству между разработчиками и клиентами отдается предпочтение перед строгими контрактами, хотя важность хорошо составленных контрактов растет такими же темпами, как и размер программного проекта. Сам процесс переговоров следует рассматривать как средство достижения и

⁴ agilemanifesto.org и www.agilealliance.org, (1.5.2002)

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Поддержание жизнеспособных отношений. С точки зрения бизнеса, гибкая разработка направлена на создание бизнес-ценности сразу после запуска проекта, что снижает риски невыполнения контракта.

В-четвертых, группа разработки, состоящая как из разработчиков программного обеспечения, так и из представителей заказчика, должна быть хорошо информирована, компетентна и уполномочена учитывать возможные потребности в корректировках, возникающие в ходе жизненного цикла процесса разработки. Это означает, что участники готовы к внесению изменений, а существующие контракты должны быть составлены с использованием инструментов, поддерживающих и позволяющих вносить эти улучшения.

По словам Хайсмита и Кокберна (2001, стр. 122), «новшество гибких методов заключается не в используемых ими практиках, а в признании людей главными движущими силами успеха проекта в сочетании с особым вниманием к эффективности и маневренности. Это создаёт новое сочетание ценностей и принципов, определяющих мировоззрение agile ». Бём (2002) иллюстрирует спектр различных методов планирования на рисунке 1, где хамеры находятся на одном конце, а так называемый «дьюймовый камешек» с железными рамками контрактного подхода — на другом:

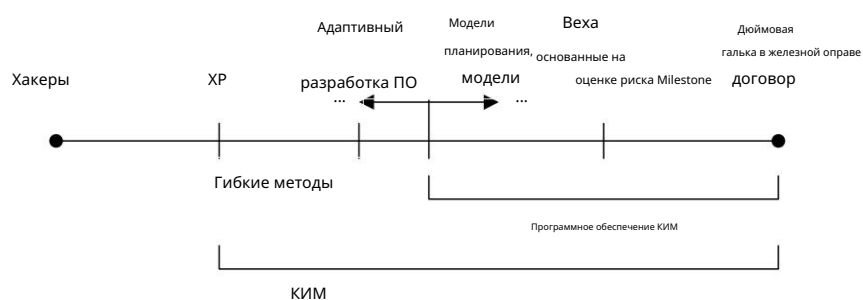


Рисунок 1. Спектр планирования (Бём 2002, стр. 65)

Хавриш и Рупрехт (2000) утверждают, что одна методология не может быть применима ко всему спектру различных проектов, поэтому руководство проекта должно определить специфику конкретного проекта и затем выбрать наиболее подходящую методологию разработки. Подчеркивая этот момент, согласно

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Макколи (2001) утверждает, что необходимы как гибкие, так и процессно-ориентированные методы, поскольку не существует универсальной модели разработки программного обеспечения, подходящей для всех мыслимых целей. Это мнение разделяют и многие эксперты в этой области (Glass, 2001).

Кокберн (2002а, стр. ххii) определяет суть гибких методов разработки программного обеспечения как использование простых, но достаточных правил поведения в проекте, а также правил, ориентированных на взаимодействие с человеком и коммуникацию. Гибкий процесс одновременно лёгкий и достаточный. Лёгкость — это способ оставаться манёвренным. Достаточность — это вопрос сохранения игры (Кокберн, 2002а). Он предлагает следующие «изюминки», наличие которых в процессе разработки программного обеспечения повышает шансы на успешный результат проекта:

- От двух до восьми человек в одной комнате

- о Коммуникация и сообщество

- Эксперты по использованию на месте

- о Короткие и непрерывные циклы обратной связи

- Короткие приращения

- о От одного до трех месяцев, позволяет быстро провести тестирование и ремонт

- Полностью автоматизированные регрессионные тесты

- о Модульные и функциональные тесты стабилизируют код и позволяют постоянное совершенствование

- Опытные разработчики

- о Опыт ускоряет время разработки с 2 до 10 раз по сравнению с более медленными членами команды.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

2.3 Характеристика

Миллер (2001) дает следующие характеристики гибким процессам разработки программного обеспечения с точки зрения быстрой поставки, которые позволяют сократить жизненный цикл проектов:

1. Модульность на уровне процесса разработки
2. Итеративный с короткими циклами, позволяющий быстро выполнять проверки и исправления.
3. Ограниченный по времени с циклами итераций от одной до шести недель
4. Экономия в процессе разработки исключает все ненужные действия.
5. Адаптивность к возможным новым рискам
6. Поэтапный подход к процессу, позволяющий функционировать приложению
строительство небольшими шагами
7. Конвергентный (и инкрементальный) подход минимизирует риски
8. Ориентированные на людей, т.е. гибкие процессы отдают предпочтение людям, а не процессам и
технология
9. Коллективный и коммуникативный стиль работы

Фаваро (2002) рассматривает появление стратегий, позволяющих справиться с неопределенным процессом, демонстрирующим меняющиеся требования. Он предлагает парадигму итеративной разработки в качестве общего знаменателя гибких процессов. Требования могут вводиться, изменяться или удаляться в последовательных итерациях. Этот подход, характеризующийся изменяющимися требованиями и отложенной реализацией, вновь требует новых договорных соглашений. К ним относятся, например, переход от контрактов с точки зрения точки зрения (с фиксированием требований заранее) к контрактам с точки зрения процесса, а также учет предвосхищения правовых аспектов в развитии отношений (Пейхёнен, 2000). Однако эти теоретические правовые аспекты все еще находятся в зачаточном состоянии, не говоря уже о конкретной капитализации этих новых договорных феноменов. Кроме того, эффективно используются рамочные контракты.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

вместе с соответствующими соглашениями о проекте или рабочем заказе отражают постоянное развитие в сфере разработки программного обеспечения, которое по своей сути поддерживает этот тип гибкой разработки программного обеспечения (Warsta 2001).

Хайсмит и Кокберн (2001) сообщают, что изменение среды в сфере разработки программного обеспечения также влияет на сами процессы разработки программного обеспечения.

По мнению авторов, удовлетворение потребностей клиентов на момент поставки стало важнее, чем удовлетворение потребностей клиентов на момент инициации проекта. Это требует разработки процедур, направленных не столько на предотвращение изменений на ранних этапах проекта, сколько на более эффективное управление неизбежными изменениями на протяжении всего его жизненного цикла. Далее утверждается, что гибкие методологии предназначены для:

- осуществить первую поставку за несколько недель, чтобы добиться ранней победы и быстрой обратной связи
- придумывать простые решения, чтобы меньше приходилось менять и делать их изменения легче
- постоянно улучшать качество дизайна, делая реализацию следующей истории менее затратной, и
- постоянно проводить испытания для более раннего и менее затратного обнаружения дефектов.

Базовые принципы гибких методов включают в себя бескомпромиссную честность рабочего кода, эффективность совместной работы людей с доброжелательностью и ориентацию на командную работу. Эмблер (2002b) предложил набор здоровых подходов, основанных на гибких процессах разработки программного обеспечения:

- люди имеют значение
- возможно меньше документации
- коммуникация является критически важным вопросом
- инструменты моделирования не так полезны, как обычно думают

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/publications/2002/P478.pdf>.

- не требуется большого предварительного проектирования.

Бём (2002) анализирует гибкие и процессно-ориентированные методологии, или, как он их называет, план-ориентированные. В таблице 2 показано, как парадигма программного обеспечения с открытым исходным кодом (ПОС) располагается между гибкими и план-ориентированными методами. ПОС всё ещё относительно новое направление в бизнес-среде, и ряд интересных исследовательских вопросов ещё предстоит проанализировать и ответить. Таким образом, подход ПОС можно рассматривать как один из вариантов многогранных гибких методов.

Таблица 2. Родная территория для гибких и плановых методов (Бём, 2002 г.)

дополнена колонкой о программном обеспечении с открытым исходным кодом.

Домашняя площадка область	Гибкие методы	Открытый исходный код программное обеспечение	Планово-ориентированные методы
Разработчики	Гибкий, знающий, умеющий работать в команде и совместный	Географически распределенный, совместный, знающие и гибкие команды	Ориентированность на план; адекватные навыки; доступ к внешним ресурсам знание
Клиенты	Преданный, знающий, совместно размещены, совместный, представитель, и уполномоченный	Преданный, знающий, совместный и уполномоченный	Доступ к знаниям, совместный, представитель, и уполномоченные клиенты
Требования	В значительной степени возникающий; быстрые изменения	В значительной степени возникающий; быстро меняющийся, находящийся в общей собственности, постоянно развивающийся – «никогда» не завершено	Познаваемый рано; в значительной степени стабильный
Архитектура	Разработано для текущих	Открыто, разработано для текущие требования	Разработано для текущих и предсказуемые требования

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Домашняя площадка область	Гибкие методы	Открытый исходный код программное обеспечение	Планово-ориентированные методы
	требования		
Рефакторинг	Недорого	Недорого	Дорогой
Размер	Меньшие команды и продукты	Более крупные рассеянные команды и меньшие продукты	Большие команды и продукты
Основная цель: быстрое получение ценности		Сложная проблема Высокая уверенность	

2.4. Резюме

Ключевые аспекты лёгких и гибких методологий — простота и скорость. Соответственно, в процессе разработки группа разработчиков концентрируется только на функциях, которые необходимы непосредственно, быстро предоставляя их, собирая обратную связь и реагируя на полученную информацию. На основе вышеизложенного предлагается определение гибкого подхода к разработке программного обеспечения, которое далее используется в данной публикации.

Что делает метод разработки гибким? Это тот случай, когда разработка программного обеспечения является инкрементальной (небольшие релизы с короткими циклами), кооперативной (заказчик и разработчики постоянно работают вместе, тесно взаимодействуя), простой (сам метод легко освоить и модифицировать, он хорошо документирован) и адаптивной (позволяет вносить изменения в последний момент).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3. Существующие гибкие методы

В этой главе рассматривается современное состояние методов гибкой разработки программного обеспечения. Выбор методов основан на определении, предложенном в разделе 2.2.

В результате в данный анализ включены следующие методы: экстремальное программирование (Бек, 1999b), Scrum (Швабер, 1995; Швабер и Бидл, 2002), семейство методологий Crystal (Кокберн, 2002a), разработка на основе функций (Палмер и Фелсинг, 2002), рациональный унифицированный процесс (Кручтен, 1996; Кручтен, 2000), метод разработки динамических систем (Стейплтон, 1997), адаптивная разработка программного обеспечения (Хайсмит, 2000) и разработка с открытым исходным кодом (например, О'Рейли, 1999).

Методы будут вводиться и рассматриваться систематически с использованием определенной структуры, в которой определены процесс, роли и обязанности, практики, принятие и опыт, область использования и текущие исследования относительно каждого метода. Процесс относится к описанию фаз жизненного цикла продукта, через которые производится программное обеспечение. Роли и обязанности относятся к распределению конкретных ролей, посредством которых осуществляется производство программного обеспечения в команде разработчиков. Практики - это конкретные виды деятельности и рабочие продукты, которые метод определяет для использования в процессе. Принятие и опыт относятся в первую очередь к существующим отчетам об опыте использования метода на практике и соображениям разработчиков метода о том, как метод должен быть внедрен в организации. Область использования определяет ограничения, касающиеся каждого метода, т. е. были ли они задокументированы. Наконец, рассматриваются текущие исследования и публикации, чтобы получить обзор научного и практического статуса метода.

Гибкое моделирование (Ambler, 2002a) и прагматическое программирование (Hunt, 2000) кратко рассматриваются в последнем разделе под названием «Другие гибкие методы». Это связано с тем, что они не являются методами как таковыми, но привлекли значительное внимание в сообществе сторонников гибких методологий и, следовательно, требуют рассмотрения.

3.1 Экстремальное программирование

Экстремальное программирование (XP) возникло из-за проблем, вызванных длительными циклами разработки традиционных моделей (Бек, 1999a). Оно впервые

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

начиналось как «просто возможность выполнить работу» (Хаунгс, 2001) с практики, которые были признаны эффективными в процессах разработки программного обеспечения В течение предыдущих десятилетий (Бек, 2000). После ряда успешных испытаний в практика (Андерсон и др., 1998), методология ХР была «теоретизирована» на ключевом Используемые принципы и практики (Бек, 2000). Хотя отдельные практики ХР не являются новыми как таковыми, в ХР они были собраны и выстроены в ряд функционируют друг с другом по-новому, формируя тем самым новую методологию Разработка программного обеспечения. Термин «экстремальный» происходит от этих принципы и практики здравого смысла доведены до крайностей (Бек, 2000).

3.1.1 Процесс

Жизненный цикл ХР состоит из пяти фаз: исследование, планирование, итерации Выпуск, выпуск в производство, обслуживание и смерть (рисунок 2).

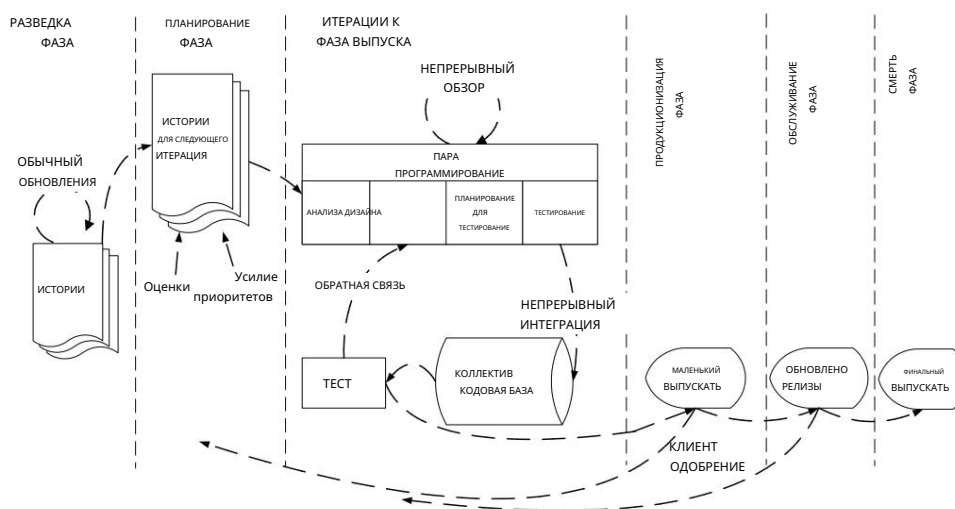


Рисунок 2. Жизненный цикл процесса ХР

Далее эти фазы представлены в соответствии с Бек (2000).
 описание:

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

На этапе исследования заказчики составляют карты историй, которые они хотели бы включить в первый релиз. Каждая карта историй описывает функцию, которую необходимо добавить в программу. В это же время команда проекта знакомится с инструментами, технологиями и методами, которые будут использоваться в проекте. Будущая технология тестируется, а возможности архитектуры системы исследуются путём создания прототипа. Этап исследования занимает от нескольких недель до нескольких месяцев, в зависимости от того, насколько хорошо программисты знакомы с технологией.

На этапе планирования устанавливается приоритет историй и согласовывается содержание первого небольшого релиза. Программисты сначала оценивают, сколько усилий потребуется для каждой истории, и согласовывают график. Срок выполнения первого релиза обычно не превышает двух месяцев. Сам этап планирования занимает пару дней.

Фаза «Итерации до выпуска» включает несколько итераций систем перед первым релизом. График, установленный на этапе планирования, разбивается на несколько итераций, реализация каждой из которых занимает от одной до четырёх недель. Первая итерация создаёт систему с архитектурой всей системы. Это достигается путём выбора историй, которые обеспечат построение структуры всей системы. Заказчик выбирает истории для каждой итерации. Функциональные тесты, созданные заказчиком, запускаются в конце каждой итерации. По завершении последней итерации система готова к производству.

Фаза подготовки к производству требует дополнительного тестирования и проверки производительности системы перед её выпуском заказчику. На этом этапе могут быть обнаружены новые изменения, и необходимо принять решение об их включении в текущую версию. На этом этапе может потребоваться сокращение итераций с трёх до одной недели. Отложенные идеи и предложения документируются для последующей реализации, например, на этапе сопровождения.

После выпуска первой версии для использования клиентом, проект XP должен поддерживать систему в рабочем состоянии, одновременно создавая новые итерации. Для этого этап сопровождения требует также затрат на поддержку клиентов. Таким образом, скорость разработки может замедлиться после

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Система находится в эксплуатации. На этапе поддержки может потребоваться включение новых сотрудников в команду и изменение её структуры.

Фаза «смерти» близка, когда у заказчика больше нет историй для реализации. Это требует, чтобы система удовлетворяла потребностям заказчика и в других отношениях (например, в отношении производительности и надежности). Это этап процесса XP, когда наконец пишется необходимая документация системы, поскольку в архитектуру, дизайн или код больше не вносятся изменения. «Смерть» также может наступить, если система не обеспечивает желаемых результатов или становится слишком дорогой для дальнейшей разработки.

3.1.2 Роли и обязанности

В XP существуют различные роли для различных задач и целей в рамках процесса и его практик. Далее эти роли представлены в соответствии с Беком (2000).

Программист .

Программисты пишут тесты и стараются сделать код программы максимально простым и понятным. Ключевым фактором успеха XP является взаимодействие и координация действий с другими программистами и членами команды.

Клиент

Заказчик пишет истории и функциональные тесты, а также решает, когда каждое требование будет выполнено. Заказчик устанавливает приоритет реализации требований.

Тестер

Тестировщики помогают заказчику писать функциональные тесты. Они регулярно проводят функциональные тесты, публикуют результаты и поддерживают инструменты тестирования.

Трекер

Трекер даёт обратную связь в XP. Он отслеживает оценки, сделанные командой (например, оценки трудозатрат), и даёт обратную связь об их точности для улучшения будущих оценок. Он также отслеживает ход каждой итерации и

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

оценивает, достижима ли цель в рамках имеющихся ресурсов и ограничений по времени или необходимы какие-либо изменения в процессе.

Тренер

Коуч — это человек, отвечающий за весь процесс. В этой роли важно глубокое понимание опыта (XP), позволяющее коучу направлять других членов команды в соответствии с процессом.

Консультант

Консультант — это внешний член команды, обладающий необходимыми техническими знаниями. Он помогает команде решать её конкретные проблемы.

Менеджер (Большой Босс)

Менеджер принимает решения. Для этого он взаимодействует с командой проекта, чтобы оценить текущую ситуацию и выявить любые трудности или недостатки в процессе.

3.1.3. Практики

XP представляет собой набор идей и практик, заимствованных из уже существующих методологий (Беск, 1999а). Структура принятия решений, представленная на рисунке 3, в которой заказчик принимает бизнес-решения, а программисты решают технические вопросы, основана на идеях Александра (1979). Стремительная эволюция типов в XP коренится в идеях, лежащих в основе Scrum⁵ (Takeuchi and Nonaka, 1986), и языке шаблонов⁶ Каннингема (Cunningham, 1996). Идея XP о планировании проектов на основе клиентских историй основана на вариантах использования⁷.

⁵ Scrum также можно рассматривать как гибкий метод. Он будет представлен в разделе 3.2.

⁶ Шаблон — это именованный фрагмент информации, раскрывающий суть проверенного решения определённой повторяющейся проблемы в определённом контексте. Язык шаблонов — это совокупность шаблонов, которые вместе помогают найти упорядоченное решение сложной проблемы, соответствующее заранее определённой цели. (Александр, 1979).

⁷ Сценарии использования используются для фиксации высокоуровневых пользовательских функциональных требований к системе (Якобсен, 1994).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

(Якобсен, 1994), а эволюционная подача заимствована у Гилба (Гилб, 1988).
 Спиральная модель, первоначальный ответ на каскадную модель, также оказала влияние на метод XP. Метафоры XP берут начало в работах Лакоффа и Джонсона (Lakoff and Johnson, 1998) и Койна (Coyne, 1995).
 Наконец, физическая рабочая среда для программистов заимствована из (Coplien 1998) и (DeMarco and Lister 1999).



Рисунок 3. Корни экстремального программирования

XP направлен на обеспечение успешной разработки программного обеспечения в небольших и средних командах, несмотря на расплывчатые или постоянно меняющиеся требования. Среди основных характеристик XP — короткие итерации с небольшими релизами и быстрой обратной связью, участие клиентов, коммуникация и координация, непрерывная интеграция и тестирование, коллективное владение кодом, ограниченное документирование и парное программирование. Практики XP представлены ниже, согласно Беку (1999а; 2000).

Игра

«Планирование». Тесное взаимодействие между заказчиком и программистами. Программисты оценивают необходимые трудозатраты на реализацию клиентских историй, а заказчик затем определяет объём и сроки релизов.

Небольшие/короткие релизы

Простая система быстро «выводится в производство» (см. рис. 2) — не реже одного раза в 2–3 месяца. Новые версии затем выпускаются даже ежедневно, но не реже раза в месяц.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Метафора.

Система определяется метафорой/набором метафор, используемых заказчиком и программистами. Эта «общая история» направляет всю разработку, описывая работу системы.

Простой дизайн.

Акцент делается на разработке максимально простого решения, реализуемого на данный момент. Ненужная сложность и лишний код сразу же удаляются.

Тестирование. Разработка программного обеспечения осуществляется через тестирование. Модульные тесты внедряются до написания кода и выполняются непрерывно. Функциональные тесты пишут заказчики.

Рефакторинг

Реструктуризация системы путем устранения дублирования, улучшения коммуникации, упрощения и добавления гибкости.

Парное программирование

Два человека пишут код на одном компьютере.

Коллективное владение.

Любой может изменить любую часть кода в любое время.

Непрерывная интеграция.

Новый фрагмент кода интегрируется в кодовую базу сразу же по мере его готовности. Таким образом, система интегрируется и собирается много раз в день. Для принятия изменений в коде выполняются все необходимые тесты, которые должны быть пройдены.

40-часовая неделя

Максимальная продолжительность рабочей недели — 40 часов. Две сверхурочные недели подряд не допускаются. В таком случае это рассматривается как проблема, требующая решения.

Клиент на месте

Клиент должен присутствовать и быть доступен для команды в течение всего рабочего дня.

Стандарты кодирования

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Существуют правила кодирования, которым следуют программисты. Следует подчеркнуть важность коммуникации посредством кода.

Открытое рабочее пространство. Предпочтительна большая комната с небольшими кабинками. Парные программисты должны располагаться в центре пространства.

Просто правила

В команде есть свои правила, которые соблюдаются, но могут быть изменены в любой момент. Изменения должны быть согласованы, а их влияние должно быть оценено.

3.1.4. Принятие и опыт

Бек (1999а, стр. 77) предполагает, что XP следует внедрять постепенно.

«Если вы хотите попробовать XP, ради всего святого, не пытайтесь проглотить всё сразу. Выберите самую серьёзную проблему в вашем текущем процессе и попробуйте решить её методом XP».

Одна из основополагающих идей XP заключается в том, что не существует процесса, подходящего для каждого проекта, а скорее практики должны быть адаптированы к потребностям каждого проекта (Beck, 2000). Вопрос в том, насколько далеко можно расширить отдельные практики и принципы, чтобы мы всё ещё могли говорить о применении XP? И насколько мы можем отказаться от них? Фактически, нет ни одного отчёта об опыте, в котором были бы внедрены все практики XP. Вместо этого, как и предполагал Бек, неоднократно сообщалось о частичном внедрении практик XP (Grenning, 2001; Schuh, 2001).

Практические подходы к внедрению XP описаны в книге «Extreme Programming Installed» (Джеффрис и др., 2001). В книге описывается набор методик, охватывающих большинство практик XP, в основном разработанных в ходе масштабного промышленного программного проекта, где использовался XP. Постоянной темой книги является оценка сложности и длительности решаемых задач. Авторы предлагают использовать пики для достижения этой цели. Пик — это короткий эксперимент в коде, используемый для понимания того, как может быть решена конкретная программная задача. Сам процесс внедрения в книге не обсуждается.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

книге, но описываемые методики снижают порог принятия, давая конкретные примеры того, каково это — на самом деле выполнять XP.

XP является наиболее документированным из различных гибких методов, и это послужило толчком к новым исследованиям, статьям и отчетам об опыте отдельных практик XP, таких как парное программирование (например, Williams et al. 2000; Haungs 2001), а также о применении самого метода. В основном сообщалось об успешном опыте (например, Anderson et al. 1998; Schuh 2001) применения XP. Эти исследования дают представление о возможностях и ограничениях XP. Maurer и Martel (2002a) приводят некоторые конкретные цифры относительно повышения производительности при использовании XP в проекте веб-разработки. Они сообщают о среднем увеличении на 66,3% количества новых строк кода, увеличении на 302,1% количества разработанных новых методов и увеличении на 282,6% количества новых классов, реализованных в процессе разработки. Более подробную информацию об использованных ими мерах и полученных результатах можно найти в (Maurer and Martel 2002b).

3.1.5 Область применения

Как утверждает Бек (2000), методология XP применима далеко не везде, и её ограничения ещё не выявлены. Это требует дополнительных эмпирических и экспериментальных исследований этой темы с разных точек зрения. Однако были выявлены некоторые ограничения.

XP предназначен для небольших и средних команд. Бек (2000) предлагает ограничить размер команды тремя и максимум двадцатью участниками проекта. Физическая среда также важна в XP. Коммуникация и координация между участниками проекта должны быть постоянно обеспечены. Например, Бек (2000) отмечает, что разбросанность программистов на двух этажах или даже на одном недопустима для XP. Однако географическое распределение команд не обязательно выходит за рамки XP, если она включает «две команды, работающие над связанными проектами с ограниченным взаимодействием» (Бек, 2000, с. 158).

Еще одним ключевым вопросом XP является деловая культура, влияющая на отдел разработки. Любое сопротивление принципам и практикам XP со стороны участников проекта, руководства или заказчика может быть достаточным для провала процесса (Beck, 2000). Кроме того, технологии могут стать непреодолимым препятствием для успеха.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Проект XP. Например, технология, которая не поддерживает «мягкие изменения» или требует длительного времени обратной связи, не подходит для процессов XP.

3.1.6 Текущие исследования

Исследования XP расширяются. Существует множество опубликованных статей, посвящённых различным аспектам XP, но, вероятно, поскольку XP рассматривается скорее как практический, чем академический метод, большинство статей посвящено опыту использования XP в различных областях и эмпирическим результатам, касающимся его применения. Некоторые из этих статей можно найти, например, в (Succi and Marchesi, 2000).

3.2. Скрам

Первые упоминания термина «Scrum» в литературе относятся к статье Такеучи и Нонаки (1986), в которой представлен адаптивный, быстрый и самоорганизующийся процесс разработки продукта, зародившийся в Японии (Швабер и Бидл, 2002). Термин «Scrum» изначально происходит от стратегии в регби, где он обозначает «возвращение мяча, выбывшего из игры, в игру» с помощью командной работы (Швабер и Бидл, 2002).

Методология Scrum была разработана для управления процессом разработки систем. Это эмпирический подход, применяющий идеи теории управления производственными процессами к разработке систем, что приводит к обновлению понятий гибкости, адаптивности и производительности (Schwaber and Beedle, 2002). Она не определяет какие-либо конкретные методы разработки программного обеспечения для его внедрения. Scrum фокусируется на том, как члены команды должны действовать, чтобы обеспечить гибкость разработки системы в постоянно меняющейся среде.

Основная идея Scrum заключается в том, что разработка систем включает в себя ряд переменных среды и технических характеристик (например, требования, сроки, ресурсы и технологии), которые могут меняться в ходе процесса. Это делает процесс разработки непредсказуемым и сложным, требуя гибкости для реагирования на изменения.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Результатом процесса разработки является система, которая полезна после поставки (Швабер, 1995).

Scrum помогает улучшить существующие инженерные практики (например, практики тестирования) в организации, поскольку он включает в себя частые управленческие действия, направленные на последовательное выявление любых недостатков или препятствий в процессе разработки, а также используемых практик.

3.2.1 Процесс

Процесс Scrum включает три фазы: предыгровую, разработку и послеигровую (рисунок 4).

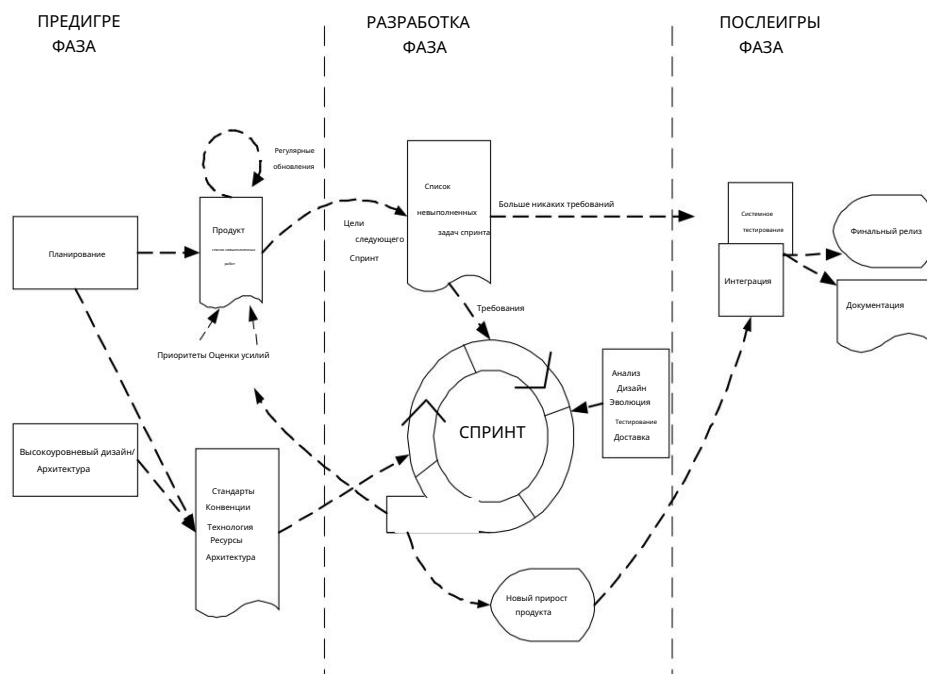


Рисунок 4. Процесс Scrum

Далее фазы Scrum будут представлены в соответствии с Швабером (1995; 2002).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Предигровая фаза включает в себя две подфазы: планирование и архитектура/высокоуровневое проектирование (рисунок 4).

Планирование включает в себя определение разрабатываемой системы. Создаётся список бэклога продукта (см. 3.2.3), содержащий все известные на данный момент требования. Требования могут исходить от заказчика, отдела продаж и маркетинга, службы поддержки клиентов или разработчиков программного обеспечения. Требования ранжируются по приоритетам, и оцениваются необходимые для их реализации усилия. Список бэклога продукта постоянно обновляется новыми и более подробными пунктами, а также более точными оценками и новыми приоритетами. Планирование также включает в себя определение команды проекта, инструментов и других ресурсов, оценку рисков и контрольных вопросов, потребности в обучении и утверждение руководства по верификации. На каждой итерации обновлённый бэклог продукта рассматривается Scrum-командой(-ами), чтобы получить их обязательства по следующей итерации.

На этапе разработки архитектуры планируется высокоуровневое проектирование системы, включая архитектуру, на основе текущих пунктов бэклога продукта. В случае усовершенствования существующей системы определяются изменения, необходимые для реализации пунктов бэклога, а также проблемы, которые они могут вызвать. Проводится совещание по обзору проекта для обсуждения предложений по реализации, и на основе этого обзора принимаются решения. Кроме того, готовятся предварительные планы содержания релизов.

Фаза разработки (также называемая игровой фазой) — это гибкая часть подхода Scrum. Эта фаза рассматривается как «чёрный ящик», где ожидается непредсказуемое. Различные внешние и технические переменные (такие как сроки, качество, требования, ресурсы, технологии и инструменты реализации, и даже методы разработки), определяемые Scrum и которые могут меняться в ходе процесса, отслеживаются и контролируются посредством различных практик Scrum в течение спринтов (см. раздел ниже) фазы разработки. Вместо того, чтобы учитывать эти факторы только в начале проекта разработки программного обеспечения, Scrum стремится контролировать их постоянно, чтобы иметь возможность гибко адаптироваться к изменениям.

На этапе разработки система разрабатывается в спринтах (рис. 4 и 3.2.3). Спринты представляют собой итерационные циклы, в которых функциональность разрабатывается или дорабатывается.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Расширены возможности создания новых инкрементов. Каждый спринт включает традиционные фазы разработки программного обеспечения: требования, анализ, проектирование, эволюцию и поставку (рис. 4). Архитектура и дизайн системы развиваются в ходе разработки спринта. Планируется, что один спринт продлится от одной недели до одного месяца. Например, в процессе разработки одной системы может быть от трёх до восьми спринтов, прежде чем система будет готова к распространению. Кроме того, над инкрементом может работать несколько команд.

Пост-игровая фаза включает в себя завершение релиза. Эта фаза начинается после достижения соглашения о том, что все переменные среды, такие как требования, выполнены. В этом случае больше не будет выявлено никаких проблем и не будет создано никаких новых. Система готова к релизу, и подготовка к нему осуществляется на пост-игровой фазе, включая такие задачи, как интеграция, тестирование системы и документирование (рис. 4).

3.2.2 Роли и обязанности

В Scrum выделяется шесть ролей, имеющих различные задачи и цели в рамках процесса и его практик: Scrum-мастер, владелец продукта, Scrum-команда, заказчик, пользователь и руководство. Ниже эти роли представлены в соответствии с определениями Швабера и Бидла (2002):

Скрам-мастер

Скрам-мастер — это новая управленческая роль, введенная Scrum. Скрам-мастер отвечает за реализацию проекта в соответствии с практиками, ценностями и правилами Scrum, а также за его реализацию по плану. Скрам-мастер взаимодействует с командой проекта, заказчиком и руководством в ходе проекта. Он также отвечает за устранение и изменение любых препятствий в процессе, чтобы команда работала максимально продуктивно.

Владелец продукта

Владелец продукта официально отвечает за проект, управляя, контролируя и предоставляя доступ к списку бэклога продукта. Он выбирается скрам-мастером, заказчиком и руководством. Он принимает окончательные решения по задачам, связанным с бэклогом продукта (см. 3.2.3), участвует в оценке

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

усилия по разработке элементов бэклога и превращение проблем в бэклоге в функции, требующие разработки.

Scrum-команда

Scrum-команда — это команда проекта, которая имеет полномочия принимать решения о необходимых действиях и организовывать свою работу для достижения целей каждого спринта. Scrum-команда участвует, например, в оценке трудозатрат, создании бэклога спринта (см. 3.2.3), анализе бэклога продукта и выработке решений по устранению препятствий, которые необходимо устранить в проекте.

Клиент

Заказчик участвует в задачах, связанных с пунктами бэклога продукта (см. 3.2.3) для разрабатываемой или совершенствуемой системы.

Руководство

отвечает за принятие окончательных решений, а также за разработку уставов, стандартов и соглашений, которые должны соблюдаться в проекте. Руководство также участвует в постановке целей и требований. Например, оно участвует в выборе владельца продукта, оценке прогресса и сокращении бэклога с помощью Scrum-мастера.

3.2.3.Практики

Scrum не требует и не предоставляет каких-либо конкретных методов/практик разработки программного обеспечения. Вместо этого он требует использования определённых управленческих практик и инструментов на различных этапах Scrum, чтобы избежать хаоса, вызванного непредсказуемостью и сложностью (Schwaber, 1995).

Далее приводится описание практик Scrum на основе работы Швабера и Бидла (2002).

Бэклог продукта

определяет всё, что требуется для конечного продукта, основываясь на текущих знаниях. Таким образом, бэклог продукта определяет работы, которые необходимо выполнить в рамках проекта. Он включает в себя приоритетный и постоянно обновляемый список бизнес- и технических требований к создаваемой или совершенствуемой системе. Элементы бэклога

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

может включать, например, функции, исправления ошибок, дефекты, запрошенные Улучшения и модернизация технологий. Также вопросы, требующие решения до Другие пункты бэклога могут быть выполнены, если они включены в список. Несколько участников могут участвовать в формировании элементов бэклога продукта, таких как клиент, проектная группа, маркетинг и продажи, управление и поддержка клиентов.

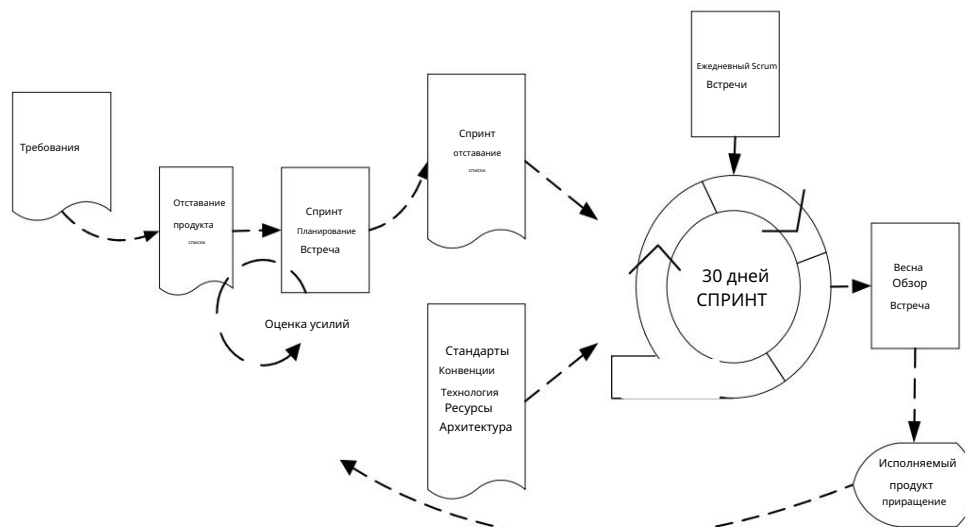
Эта практика включает в себя задачи по созданию списка невыполненных работ по продукту, а также контролируя его последовательно в ходе процесса путем добавления, удаления, уточнения, Обновление и приоритизация элементов бэклога продукта. Владелец продукта — отвечает за ведение журнала невыполненных работ по продукту.

Оценка усилий

Оценка усилий — это итеративный процесс, в котором оцениваются элементы бэклога. ориентированы на более точный уровень, когда доступно больше информации о определённый элемент бэклога продукта. Владелец продукта совместно со Scrum Команда(ы) несут ответственность за оценку усилий.

Спринт

Спринт – это процедура адаптации к изменяющимся переменным окружающей среды. (требования, время, ресурсы, знания, технологии и т. д.). Scrum-команда самоорганизуется для создания нового исполняемого продукта в спринте, который Продолжительность работы составляет около тридцати календарных дней. Рабочие инструменты команды: Встречи по планированию спринта, бэклог спринта и ежедневные встречи Scrum (см. (см. ниже). Спринт с его практиками и входными данными показан на рисунке 5.



Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Рисунок 5. Практики и вклады спринта

Совещание по планированию

спринта. Совещание по планированию спринта — это двухэтапное совещание, организованное скрам-мастером. В первом этапе совещания участвуют заказчики, пользователи, руководство, владелец продукта и скрам-команда, чтобы определить цели и функционал следующего спринта (см. бэклог спринта ниже). Второй этап совещания проводится скрам-мастером и скрам-командой, где основное внимание уделяется реализации инкремента продукта в ходе спринта.

Бэклог спринта.

Бэклог спринта — это отправная точка каждого спринта. Это список элементов бэклога продукта, выбранных для реализации в следующем спринте. Элементы выбираются Scrum-командой совместно со Scrum-мастером и владельцем продукта на совещании по планированию спринта на основе приоритетов (см. 3.2.3) и целей, установленных для спринта. В отличие от бэклога продукта, бэклог спринта остаётся неизменным до завершения спринта (т.е. 30 дней). После завершения всех элементов бэклога спринта предоставляется новая итерация системы.

Ежедневные встречи

Scrum -команды организуются для непрерывного отслеживания прогресса Scrum-команды и служат также совещаниями по планированию: что было сделано с момента предыдущей встречи и что необходимо сделать к следующей. На этих коротких (примерно 15 минут) встречах, проводимых ежедневно, обсуждаются и контролируются проблемы и другие переменные вопросы. Любые недостатки или препятствия в процессе разработки систем или инженерных практиках выявляются, выявляются и устраняются для улучшения процесса. Scrum-встречи проводит Scrum-мастер. Помимо Scrum-команды, в них может участвовать, например, руководство.

Обзорная встреча по

спринту. В последний день спринта Scrum-команда и Scrum-мастер представляют результаты спринта (т.е. рабочий инкремент продукта) руководству, клиентам, пользователям и владельцу продукта на неформальной встрече. Участники оценивают инкремент продукта и принимают решения по следующим действиям. Обзорная встреча может выявить новые элементы бэклога и даже изменить направление разработки системы.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.2.4. Принятие и опыт

Поскольку Scrum не требует применения каких-либо конкретных инженерных практик, его можно адаптировать для управления любыми инженерными практиками, используемыми в организации. (Швабер и Бидл, 2002). Однако Scrum может существенно изменить должностные инструкции и традиции команды проекта Scrum. Например, теперь не менеджер проекта, то есть Scrum-мастер, организует команду, а команда организуется сама и принимает решения о дальнейших действиях. Такую команду можно назвать самоорганизующейся (Швабер и Бидл, 2002). Вместо этого Scrum-мастер работает над устранением препятствий в процессе, проводит ежедневные Scrum-встречи и принимает решения, а затем согласовывает их с руководством. Теперь его роль скорее напоминает роль тренера, чем менеджера проекта.

Швабер и Бидл (2002) выделяют два типа ситуаций, в которых возможно внедрение Scrum: существующий проект и новый проект. Они описаны ниже. Типичный случай внедрения Scrum в существующем проекте — это ситуация, когда среда разработки и используемая технология уже существуют, но команда проекта сталкивается с проблемами, связанными с меняющимися требованиями и сложной технологией. В этом случае внедрение Scrum начинается с ежедневных Scrum-сессий со Scrum-мастером. Целью первого спринта должна быть «демонстрация любой пользовательской функциональности на выбранной технологии». (Швабер и Бидл, 2002, стр. 59). Это поможет команде поверить в себя, а заказчику – в команду. Во время ежедневных скрамов первого спринта выявляются и устраняются препятствия на пути проекта, что позволяет команде двигаться вперед. В конце первого спринта заказчик и команда вместе со скрам-мастером проводят обзор спринта и совместно решают, куда двигаться дальше. В случае продолжения проекта проводится совещание по планированию спринта для определения целей и требований к следующему спринту.

При внедрении Scrum в новый проект Швабер и Бидл (2002) предлагают сначала поработать с командой и заказчиком в течение нескольких дней, чтобы сформировать начальный бэклог продукта. На этом этапе бэклог продукта может включать в себя требования к бизнес-функциональности и технологии. Цель первого спринта — «продемонстрировать ключевой элемент пользовательской функциональности на выбранной технологии» (Швабер и Бидл, 2002, стр. 59). Для этого, естественно, сначала требуется спроектировать и создать начальную системную структуру, то есть рабочую структуру для создаваемой системы, в которую можно добавлять новые функции. Спринт

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Бэклог должен включать задачи, необходимые для достижения цели спринта. Поскольку основной вопрос на данном этапе касается внедрения Scrum, бэклог спринта также включает задачи по формированию команды и ролей Scrum, а также по выстраиванию управленческих практик, помимо непосредственно реализации демоверсии. Пока команда работает с бэклогом спринта, владелец продукта совместно с заказчиком работает над созданием более полного бэклога продукта, чтобы можно было спланировать следующий спринт после проведения первого обзора спринта.

Райзинг и Яноф (2000) сообщают об успешном опыте использования Scrum в трёх проектах по разработке программного обеспечения. Они отмечают: «Очевидно, что [Scrum] не подходит для больших и сложных командных структур, но мы обнаружили, что даже небольшие, изолированные команды, работающие над крупным проектом, могут использовать некоторые элементы Scrum». Это настоящее разнообразие процессов». Необходимо чётко определить взаимодействие между небольшими подкомандами. Лучшие результаты были достигнуты благодаря адаптации некоторых принципов Scrum, таких как ежедневные встречи Scrum. Команды в случаях Rising и Janof пришли к выводу, что двух-трёх встреч в неделю достаточно для соблюдения графика спринта. Среди других положительных моментов можно отметить, например, рост волонтерства в команде и более эффективное решение проблем.

3.2.5 Область применения

Методология Scrum подходит для небольших команд, состоящих менее чем из 10 инженеров. Швабер и Бидл предлагают, чтобы команда состояла из пяти-девяти участников проекта (2002). Если доступно больше людей, следует сформировать несколько команд.

3.2.6 Текущие исследования

В последнее время наблюдаются попытки интегрировать XP и Scrum⁸. Scrum рассматривается как фреймворк управления проектами, поддерживаемый практиками XP, для формирования интегрированного пакета для команд разработки программного обеспечения. Авторы утверждают, что это повышает масштабируемость XP для более крупных проектов. Однако исследований, подтверждающих их аргументы, не существует.

⁸ xp@Scrum, www.controlchaos.com/xpScrum.htm, 16.8.2002

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.3. Семейство методологий Crystal

Семейство методологий Crystal включает в себя ряд различных методик, позволяющих выбрать наиболее подходящую для каждого проекта. Помимо самих методологий, подход Crystal также включает принципы адаптации методологий к различным условиям различных проектов.

Каждый элемент семейства Crystal отмечен цветом, указывающим на «тяжесть» методологии, т. е. чем темнее цвет, тем тяжелее методология. Crystal предлагает выбирать подходящий цвет методологии для проекта на основе его размера и критичности (рисунок 6). Более крупные проекты, вероятно, потребуют большей координации и более тяжелых методологий, чем небольшие. Чем критичнее разрабатываемая система, тем большая строгость требуется. Символы на рисунке 6 указывают на потенциальные потери, вызванные сбоем системы (т. е. уровень критичности): Комфорт (C), Дискреционные деньги (D), Необходимые деньги (E) и Жизнь (L) (Cockburn 2002a). Другими словами, уровень критичности C указывает на то, что сбой системы из-за дефектов приводит к потере комфорта для пользователя, тогда как дефекты в жизненно важной системе могут буквально привести к потере жизни.

Измерения критичности и размера представлены символом категории проекта, описанным на рисунке 6; так, например, D6 обозначает проект с максимум шестью людьми, создающими систему максимальной критичности дискреционных денег.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.



Рисунок 6. Измерения методологий кристалла (Кокберн, 2002а)

Существуют определённые правила, особенности и ценности, общие для всех методов семейства Crystal. Прежде всего, проекты всегда используют инкрементальные циклы разработки с максимальной продолжительностью инкремента в четыре месяца, но предпочтительно от одного до трёх месяцев (Cockburn, 2002a). Акцент делается на коммуникации и сотрудничестве между людьми. Методологии Crystal не ограничивают какие-либо методы разработки, инструменты или рабочие продукты, допуская при этом внедрение, например, XP и Scrum (Cockburn, 2002a).

Кроме того, подход Crystal воплощает цели по сокращению промежуточных рабочих продуктов и формированию условностей в методологии для отдельных проектов и их развитию по мере развития проекта.

В настоящее время разработаны три основные методологии Crystal: Crystal Clear, Crystal Orange и Crystal Orange Web (Cockburn, 2002a). Первые две из них были апробированы на практике и поэтому также представлены и обсуждаются в этом разделе.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.3.1 Процесс

Все методологии семейства Crystal содержат рекомендации по стандартам политики, рабочим продуктам, «местным вопросам», инструментам, стандартам и ролям (см. 3.3.2), которым необходимо следовать в процессе разработки. Crystal Clear и Crystal Orange – два представителя семейства Crystal, разработанные и используемые (Cockburn, 1998; Cockburn, 2002a). Crystal Orange (Cockburn, 1998) также описывает виды деятельности, входящие в этот процесс. В этом подразделе эти две методологии рассматриваются с учетом их контекста, сходств и различий, а также иллюстрируются процесс и деятельность Crystal Orange.

Crystal Clear разработан для очень небольших проектов (проектов категории D6), в которых задействовано до шести разработчиков. Однако, с некоторыми дополнениями для коммуникации и тестирования, его можно применять и для проектов категории E8/D10. Команда, использующая Crystal Clear, должна работать в общем офисе из-за ограничений, накладываемых его коммуникационной структурой (Cockburn, 2002a).

Crystal Orange предназначен для проектов среднего размера с общим числом участников от 10 до 40 (категория D40) и продолжительностью проекта от одного до двух лет. Проекты категории E50 также могут быть применимы в Crystal Orange с дополнениями к процессам верификации и тестирования (Cockburn, 2002a). В Crystal Orange проект разделяется на несколько команд с кросс-функциональными группами (см. 3.3.2) с использованием стратегии целостного разнообразия (см. 3.3.3). Тем не менее, этот метод не поддерживает распределенную среду разработки. Crystal Orange подчеркивает важность вопроса времени выхода на рынок. Компромисс между обширными результатами и быстрым изменением требований и дизайна приводит к ограниченному количеству результатов, что позволяет снизить затраты на их поддержку, сохраняя при этом эффективную коммуникацию между командами (Cockburn, 1998).

Далее рассматриваются сходства и различия между методами Crystal Clear и Orange с точки зрения различных элементов, предоставляемых семейством Crystal для каждого из его компонентов. Роли более подробно рассматриваются в подразделе 3.3.2.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Стандарты политики

Стандарты политики – это практики, которые необходимо применять в процессе разработки. Как Crystal Clear, так и Crystal Orange предлагают следующие стандарты политики (Cockburn, 2002a):

- Поэтапная поставка на регулярной основе
- Отслеживание прогресса по контрольным точкам на основе поставок программного обеспечения и основных решения, а не письменные документы
- Прямое участие пользователей
- Автоматизированное регрессионное тестирование функциональности
- Два просмотра пользователем на каждый релиз
- Семинары по настройке продукта и методологии в начале и в процессе середины каждого приращения

Единственное различие между стандартами политики этих двух методологий заключается в том, что Crystal Clear предполагает поэтапную поставку в течение двух-трех месяцев, тогда как в Crystal Orange поэтапный платеж может быть продлен максимум до четырех месяцев.

Стандарты политики этих методологий Crystal являются обязательными, но, тем не менее, могут быть заменены эквивалентными практиками других методологий, таких как XP или Scrum (Cockburn 2002a).

Рабочие продукты

Требования Кокберна (2002a) к рабочим продуктам Crystal Clear и Crystal Orange в некоторой степени различаются. Схожие рабочие продукты Crystal Clear и Orange включают следующие элементы: последовательность выпуска, общие объектные модели, руководство пользователя, тестовые случаи и код миграции.

Кроме того, Crystal Clear включает аннотированные описания вариантов использования и функций, тогда как в Crystal Orange требуется документ с требованиями.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

В Clear график документируется только по просмотрам и поставкам пользователями, а аналогичный рабочий продукт в Orange, который упоминает Кокберн (2002а), называется «графиком», что указывает на необходимость более детального планирования проекта. Более лёгкий характер рабочих продуктов в Crystal Clear проявляется и в проектной документации: в то время как Orange требует документы по дизайну пользовательского интерфейса и межкомандные спецификации, в Clear при необходимости предлагаются только черновики экранов, эскизы дизайна и заметки. Помимо этих рабочих продуктов, Crystal Orange требует отчёты о статусе.

Местные вопросы

Локальные вопросы – это процедуры Crystal, которые необходимо применять, но их реализация остаётся на усмотрение самого проекта. Локальные вопросы Crystal Clear и Crystal Orange существенно не отличаются друг от друга. Обе методологии предполагают, что шаблоны для рабочих продуктов, а также стандарты кодирования, регрессионного тестирования и пользовательского интерфейса должны устанавливаться и поддерживаться самой командой (Cockburn, 2002а). Например, проектная документация обязательна, но её содержание и форма остаются локальным вопросом. Кроме того, ни Crystal Clear, ни Crystal Orange не определяют методы, которые должны использоваться отдельными ролями в проекте.

Инструменты

Инструменты, необходимые для методологии Crystal Clear, включают компилятор, систему управления версиями и конфигурацией, а также печатные доски (Cockburn, 2002а). Печатные доски используются, например, для замены формальных проектных документов и сводок совещаний. Другими словами, они используются для хранения и представления материалов, которые в противном случае пришлось бы печатать в формате официальных документов после совещания.

Минимальный набор инструментов, необходимых Crystal Orange, включает в себя инструменты для управления версиями, программирования, тестирования, коммуникации, отслеживания проекта, рисования и измерения производительности. Кроме того, для повторяемых тестов графического интерфейса необходимы драйверы экрана (Cockburn, 1998).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Стандарты

Crystal Orange предлагает выбрать стандарты обозначений, соглашения по дизайну, Стандарты форматирования и стандарты качества (Кокберн, 1998) для использования в проект.

Деятельность

Более подробно деятельность Crystal Orange обсуждается в разделе 3.3.3. Однако они включены в рисунок 7 как часть иллюстрации одного приращение процесса Кристалл.

ОДНО ПРИРАЩЕНИЕ

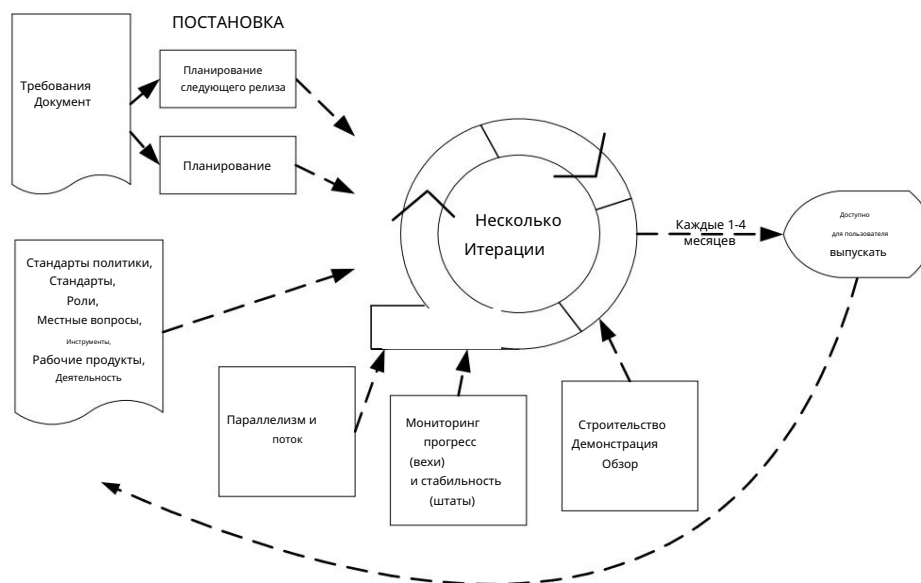


Рисунок 7. Один кристалл оранжевого цвета

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.3.2 Роли и обязанности

В этом разделе роли в Crystal Clear и Crystal Orange представлены в соответствии с методологией Кокберна (2002a). Основное различие между Crystal Clear и Orange заключается в том, что в первом случае в проекте участвует только одна команда. В Crystal Orange в проекте участвуют несколько команд. В обеих методологиях одно задание может включать несколько ролей.

В Crystal Clear основными ролями, требующими отдельных лиц, являются (Cockburn, 2002a): спонсор, старший дизайнер-программист, дизайнер-программист и пользователь. Эти роли включают в себя несколько подролей. Например, дизайнер-программист может быть одновременно дизайнером бизнес-класса, программистом, составителем документации и тестировщиком модулей (Cockburn, 1998). Подроли, которые могут быть назначены другим ролям, — это координатор, бизнес-эксперт и сборщик требований (Cockburn, 2002a). Бизнес-эксперт представляет бизнес-представление в проекте, обладая знаниями о конкретном бизнес-контексте. Он должен уметь разрабатывать бизнес-план, обращая внимание на то, что стабильно, а что меняется (Cockburn, 1998).

В дополнение к ролям, представленным в Crystal Clear, Crystal Orange предлагает широкий спектр основных ролей, необходимых для проекта. Роли сгруппированы в несколько команд, таких как команды системного планирования, наставничества проекта, архитектуры, технологий, функций, инфраструктуры и внешнего тестирования (Cockburn, 2002a). Команды далее делятся на кросс-функциональные группы, содержащие различные роли. (Кокберн, 2002a).

Crystal Orange вводит несколько новых ролей (Cockburn, 1998; Cockburn, 2002a), таких как дизайнер пользовательского интерфейса, дизайнер баз данных, эксперт по использованию, технический координатор, бизнес-аналитик/дизайнер, архитектор, наставник по дизайну, специалист по повторному использованию, писатель и тестировщик. Crystal Orange (Cockburn, 1998) также включает навыки и методы, необходимые для успешного выполнения этих ролей. Один участник проекта может совмещать несколько ролей. Например, роль специалиста по повторному использованию — это временная роль, которую может выполнять архитектор или дизайнер-программист (Cockburn, 1998). Эта роль связана с выявлением повторно используемых программных компонентов и приобретением коммерческих компонентов. Другие роли, требующие дальнейшего разъяснения, — это писатель и бизнес-аналитик-дизайнер. Роль писателя включает в себя создание внешней документации, такой как спецификации экранов и руководства пользователя.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

(Кокберн, 1998). Бизнес-аналитик-проектировщик общается и ведёт переговоры с пользователями, чтобы определить требования и интерфейсы, а также проанализировать проект.

Каждая группа состоит как минимум из дизайнера бизнес-аналитика, дизайнера пользовательского интерфейса, двух-трёх дизайнеров-программистов, дизайнера баз данных и, возможно, тестировщика. Также в группу могут быть включены технические координаторы. Создание кросс-функциональных групп обусловлено необходимостью сокращения объёма работ и улучшения локальной коммуникации. (Кокберн 2002а)

3.3.3. Практики

Все методологии Crystal включают ряд практик, таких как инкрементальная разработка. В описании Crystal Orange (Cockburn, 1998) инкрементальная разработка включает такие виды деятельности, как этапирование, мониторинг, анализ, а также параллелизм и поток (рис. 7). Также можно выделить другие виды деятельности и практики. Они включены в следующие описания.

Подготовка включает в себя планирование следующего этапа разработки системы. Выпуск рабочей версии должен быть запланирован максимум каждые три-четыре месяца (Cockburn, 1998). Предпочтительным является график сроком от одного до трёх месяцев (Cockburn, 2002а). Команда выбирает требования для реализации в рамках этапа и составляет план того, что, по её мнению, она может реализовать (Cockburn, 1998).

Пересмотр и обзор

Каждый инкремент включает несколько итераций. Каждая итерация включает следующие действия: конструирование, демонстрацию и обзор целей инкремента (Cockburn, 1998).

Мониторинг.

Прогресс отслеживается в отношении результатов работы команды в процессе разработки с точки зрения их прогресса и стабильности (Cockburn, 1998). Прогресс измеряется вехами (начало, обзор 1, обзор 2, тестирование,

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

(доставки) и стадии стабильности (значительно колеблющиеся, колеблющиеся и достаточно стабильные для анализа). Мониторинг требуется как для Crystal Clear, так и для Crystal Orange.

Параллелизм и поток

Как только мониторинг стабильности даёт результат, «достаточно стабильный для проверки» результатов, можно приступить к следующей задаче. В Crystal Orange это означает, что несколько команд могут успешно работать с максимальным параллелизмом. Для этого команды мониторинга и архитектуры анализируют свои рабочие планы, стабильность и синхронизацию (Cockburn, 1998).

Целостная стратегия

разнообразия. Crystal Orange использует метод, называемый целостной стратегией разнообразия, для разделения крупных функциональных команд на кросс-функциональные группы. Основная идея заключается в объединении специалистов разных специализаций в одну команду (Cockburn, 1998, стр. 214). Целостная стратегия разнообразия также позволяет формировать небольшие команды с необходимыми специальными знаниями и навыками, а также учитывает такие вопросы, как размещение команд, коммуникация и документирование, а также координация работы нескольких команд. (Кокберн, 1998).

Методика настройки методологии.

Методика настройки методологии — одна из базовых методик Crystal Clear и Orange. Она использует проектные интервью и командные семинары для разработки конкретной методологии Crystal для каждого отдельного проекта (Cockburn, 2002a). Одна из центральных идей инкрементальной разработки заключается в возможности исправления или улучшения процесса разработки (Cockburn, 1998). На каждом инкременте проект может учиться и использовать полученные знания для разработки процесса для следующего инкремента.

Просмотры

пользователей. Для Crystal Clear рекомендуется два просмотра на каждый релиз (Cockburn, 2002a). В Crystal Orange обзоры пользователей следует организовывать три раза для каждого выпуска (Cockburn, 1998).

Семинары по рефлексии. В

Crystal Clear и Orange есть правило, согласно которому команда должна проводить семинары по рефлексии до и после этапа приращения (с рекомендацией проводить также семинары по рефлексии в середине этапа) (Cockburn 2002a).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Crystal Clear и Crystal Orange не определяют какие-либо конкретные практики или методы, которые должны использоваться участниками проекта при разработке программного обеспечения. В Crystal допускается использование практик из других методологий, таких как XP и Scrum, для замены некоторых собственных практик, например, семинаров-размышлений.

3.3.4. Принятие и опыт

Существует как минимум один отчет об опыте (Cockburn, 1998) проекта, в котором были внедрены, развиты и использованы практики, составляющие в настоящее время методологию Crystal Orange (Cockburn, 2002b). Двухлетний «проект Winifred» был проектом среднего размера с 20–40 сотрудниками, с поэтапной стратегией разработки, объектно-ориентированным подходом и целью создания переписанной устаревшей мэйнфреймовой системы.

Согласно Кокберну (1998), проект Winifred столкнулся с проблемами на первом этапе. Проблемы с коммуникацией внутри и между командами, длительный этап требований, который не позволял разработать какую-либо архитектуру в течение нескольких месяцев, высокая текучесть кадров среди технических руководителей и нечеткое распределение обязанностей привели к тому, что процесс разработки программного обеспечения принял форму, которая сейчас называется Crystal Orange. Одним из решений стало принятие итеративного процесса для каждого этапа. Это дало командам возможность выявить свои слабые стороны и реорганизоваться. Кроме того, итерации включали функциональные просмотры с пользователями для определения окончательных требований. Это поддерживало постоянную вовлеченность пользователей в проект. Более того, уроки, извлеченные из первого этапа, побудили участников сосредоточиться на таких вопросах, как четкие распределения обязанностей, планирование каналов коммуникации, определение ответственности за результаты и поддержка руководства.

Следует отметить, что в проекте «Уинифред» разные команды могли иметь разное количество итераций в зависимости от поставленных задач. Взаимодействие между командами, например, в виде результатов, затем планировалось в соответствии с итерациями, и наоборот.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

В заключение следует отметить, что ключевыми факторами успеха проекта Уинифред были этапы, позволившие отладить процесс, определенные ключевые лица и организация команды, выработавшая привычку к выполнению поставленных задач (Кокберн, 1998).

3.3.5. Область применения

В настоящее время семейство методологий Crystal не охватывает жизненно важные проекты (рис. 6, Кокберн, 2002а). Ещё одно ограничение, выявленное Кокберном (2002а), заключается в том, что эти методологии могут применяться только к командам, работающим в одном месте.

Кокберн (2002а) также выявил ограничения, касающиеся отдельных методологий, используемых в семействе Crystal. Например, Crystal Clear имеет относительно ограниченную структуру коммуникации и поэтому подходит только для одной команды, работающей в одном офисе. Кроме того, в Crystal Clear отсутствуют элементы системной валидации, поэтому он не подходит для систем жизнеобеспечения.

Crystal Orange также требует, чтобы его команды располагались в одном офисном здании, и может предоставлять только системный максимум дискреционных средств в рамках критичности. Кроме того, у компании отсутствует структура подгрупп, что делает её подходящей для проектов с участием до 40 человек. Кроме того, компания недостаточно компетентна в вопросах проектирования и верификации кода и, следовательно, не подходит для жизненно важных систем.

3.3.6 Текущие исследования

Хотя существуют только две из четырёх предложенных методологий Crystal, Алистер Кокберн⁹ продолжает развивать своё семейство методологий Crystal, охватывающее различные типы проектов и проблемные области. За пределами этого исследования не обнаружено.

⁹ crystallmethodologies.org

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.4. Разработка на основе функций

Разработка на основе функций (FDD) — это гибкий и адаптивный подход к разработке систем. FDD-подход не охватывает весь процесс разработки программного обеспечения, а фокусируется на этапах проектирования и разработки (Palmer and Felsing, 2002). Тем не менее, он был разработан для взаимодействия с другими этапами проекта разработки программного обеспечения (Palmer and Felsing, 2002) и не требует использования какой-либо конкретной модели процесса. FDD-подход воплощает итеративную разработку с использованием лучших практик, доказавших свою эффективность в отрасли. Он уделяет особое внимание аспектам качества на протяжении всего процесса и включает в себя частые и ощутимые поставки, а также точный мониторинг хода проекта.

FDD состоит из пяти последовательных процессов и предоставляет методы, методики и рекомендации, необходимые заинтересованным сторонам проекта для предоставления системы. Кроме того, FDD включает роли, артефакты, цели и сроки, необходимые для проекта (Palmer and Felsing, 2002). В отличие от некоторых других гибких методологий, FDD, как утверждается, подходит для разработки критически важных систем (Palmer and Felsing, 2002).

FDD впервые был описан в работе (Coad et al., 2000). Затем он получил дальнейшее развитие на основе работы, проделанной для крупного проекта по разработке программного обеспечения Джеффом Лукой, Питером Коадом и Стивеном Палмером.

3.4.1 Процесс

Как упоминалось ранее, FDD состоит из пяти последовательных процессов, в ходе которых осуществляется проектирование и разработка системы (рис. 8). Итеративная часть процессов FDD (проектирование и разработка) поддерживает гибкую разработку с быстрой адаптацией к поздним изменениям требований и бизнес-потребностей. Как правило, итерация функции занимает от одной до трёх недель.

команда.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

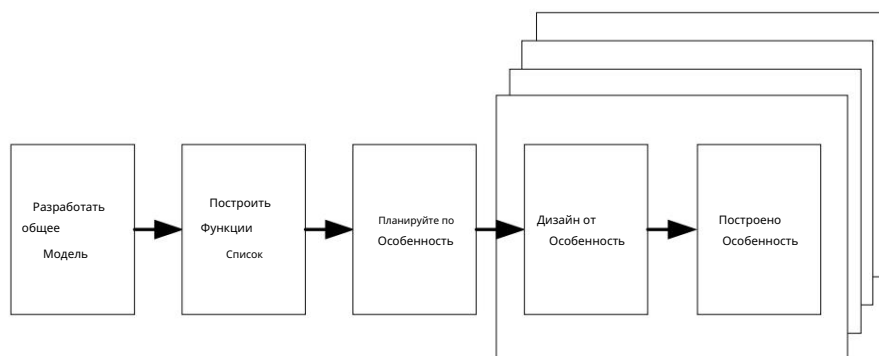


Рисунок 8. Процессы FDD (Палмер и Фелсинг, 2002)

Далее каждый из пяти процессов описывается согласно Палмеру (2002).

Разработка общей модели.

Когда начинается разработка общей модели, эксперты в предметной области (см. 3.4.2) уже знают область применения, контекст и требования создаваемой системы (Palmer and Felsing, 2002). На этом этапе, вероятно, уже существуют документированные требования, такие как варианты использования или функциональные спецификации. Однако FDD не рассматривает вопрос сбора и управления требованиями напрямую. Эксперты в предметной области представляют так называемый «пошаговый обзор», в котором члены команды и главный архитектор информируются об общем описании системы.

Общий домен далее делится на различные доменные зоны и более

Участники домена проводят подробный анализ каждого из них. После каждого анализа команда разработчиков работает в небольших группах над созданием объектных моделей для рассматриваемой предметной области. Затем команда разработчиков обсуждает и выбирает подходящие объектные модели для каждой предметной области. Одновременно формируется общая структура модели системы.

(Палмер и Фелсинг, 2002).

Составьте список функций

Пошаговые руководства, объектные модели и существующая документация по требованиям служат хорошей основой для составления полного списка функций разрабатываемой системы. В этом списке команда разработчиков представляет каждую из функций, важных для клиента, включённых в систему. Функции представлены для каждой из

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Доменные области и эти функциональные группы состоят из так называемых основных наборов функций. Кроме того, основные наборы функций подразделяются на наборы функций, которые представляют различные виды деятельности в рамках конкретных областей. Список функций проверяется пользователями и спонсорами системы на предмет его корректности и полноты.

Планирование по

функциям. Планирование по функциям включает в себя создание плана высокого уровня, в котором наборы функций упорядочены в соответствии с их приоритетом и зависимостями и назначены главным программистам (см. 3.4.2). Кроме того, классы, определенные в процессе «разработки общей модели», назначаются отдельным разработчикам, то есть владельцам классов (см. 3.4.2). Также для наборов функций могут быть установлены график и основные этапы.

Проектирование по функциям и сборка по функциям

Небольшая группа функций выбирается из набора(ов) функций, и команды по функциям, необходимые для разработки выбранных функций, формируются владельцами класса. Процессы проектирования по функциям и сборки по функциям являются итеративными процедурами, в ходе которых производятся выбранные функции. Одна итерация должна занимать от нескольких дней до максимум двух недель. Может быть несколько команд по функциям (см. 3.4.2), одновременно проектирующих и создающих свой собственный набор функций. Этот итеративный процесс включает в себя такие задачи, как проверка дизайна, кодирование, модульное тестирование, интеграция и проверка кода. После успешной итерации завершённые функции перемещаются в основную сборку, в то время как итерация проектирования и сборки начинается с новой группы функций, взятых из набора функций (см. Рисунок 9).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

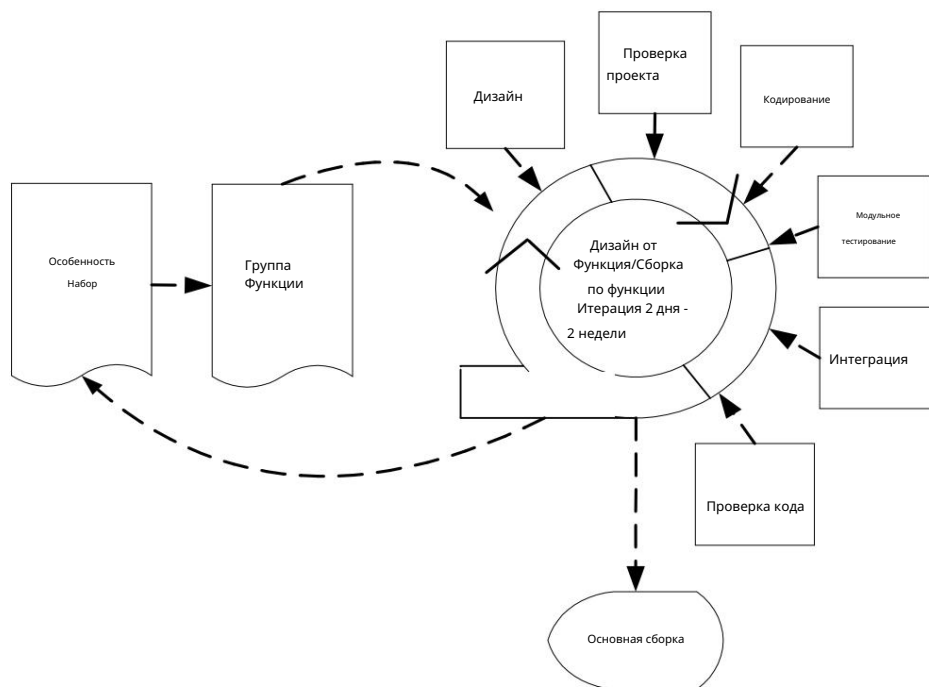


Рисунок 9. Процессы проектирования по функциям и сборки по функциям FDD

3.4.2 Роли и обязанности

FDD классифицирует свои роли по трём категориям: ключевые роли, вспомогательные роли и дополнительные роли (Palmer and Felsing, 2002). Шесть ключевых ролей в проекте FDD: менеджер проекта, главный архитектор, менеджер разработки, главный программист, владелец класса и эксперты в предметной области. Пять вспомогательных ролей включают менеджера по выпуску, юриста/гуру по языку, инженера по сборке, инструменталиста и системного администратора. Три дополнительные роли, необходимые в любом проекте, — это тестировщики, развёртыватели и технические писатели. Один член команды может играть несколько ролей, и одна роль может быть разделена между несколькими людьми (Palmer and Felsing, 2002).

Далее кратко описываются задачи и обязанности различных ролей, представленные Палмером (2002).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/publications/2002/P478.pdf>.

Менеджер проекта

(Project Manager) — это административный и финансовый руководитель проекта. Одна из его задач — защищать команду проекта от внешних отвлекающих факторов и обеспечивать её совместную работу, обеспечивая ей необходимые условия труда. В рамках FDD менеджер проекта принимает окончательное решение относительно объёма, графика и кадрового обеспечения проекта.

Главный архитектор

Главный дизайнер отвечает за общее проектирование системы и проводит рабочие совещания по проектированию с командой (Palmer and Felsing, 2002). Главный архитектор также принимает окончательные решения по всем вопросам проектирования. При необходимости эта роль может быть разделена на роли архитектора предметной области и технического архитектора.

Менеджер по разработке.

Менеджер по разработке руководит повседневной деятельностью по разработке и разрешает любые конфликты, возникающие внутри команды. Кроме того, эта роль включает в себя ответственность за решение проблем с ресурсами. Задачи этой роли могут совмещаться с обязанностями главного архитектора или менеджера проекта.

Главный программист.

Главный программист — опытный разработчик, участвующий в анализе требований и проектировании проектов. Главный программист отвечает за руководство небольшими командами, занимающимися анализом, проектированием и разработкой новых функций. В обязанности главного программиста также входит выбор функций из набора(ов) функций для разработки в следующей итерации процесса «проектирование по функциям и сборка по функциям», а также определение классов и владельцев классов, необходимых команде разработки функций для данной итерации. Он также сотрудничает с другими главными программистами в решении технических и ресурсных вопросов, а также еженедельно отчитывается о ходе работы команды.

Владелец класса

Владельцы классов работают под руководством главного программиста, занимаясь проектированием, кодированием, тестированием и документированием. Он отвечает за разработку класса, владельцем которого он назначен. Владельцы классов формируют команды разработки функций. В каждой итерации участвуют владельцы классов, чьи класс включен в функции, выбранные для следующей итерации разработки.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Эксперт в предметной

области. Эксперт в предметной области может быть пользователем, клиентом, спонсором, бизнес-аналитиком или комбинацией этих двух категорий. Его/ее задача — обладать знаниями о том, как должна выполняться работа в соответствии с различными требованиями к разрабатываемой системе. Эксперты в предметной области передают эти знания разработчикам, чтобы гарантировать создание ими качественной системы.

Менеджер домена

Менеджер домена руководит экспертами домена и разрешает разногласия между ними относительно требований к системе.

Менеджер по релизам

контролирует ход процесса, анализируя отчеты ведущих программистов о ходе работ и проводя с ними короткие совещания. Он отчитывается о ходе работ перед руководителем проекта.

Юрист по языкам/лингвистический гуру —

член команды, ответственный за глубокое знание, например, конкретного языка программирования или технологии. Эта роль особенно важна, когда команда проекта работает с новой технологией.

Инженер по сборке

Лицо, ответственное за настройку, поддержку и запуск процесса сборки, включая задачи управления системой контроля версий и публикацию документации.

Инструментальщик

Роль Toolsmith заключается в создании небольших инструментов для команд разработки, тестирования и преобразования данных в рамках проекта. Toolsmith также может заниматься настройкой и поддержкой баз данных и веб-сайтов для конкретных целей проекта.

Системный администратор.

Задача системного администратора — настраивать, управлять и устранять неполадки серверов, сети рабочих станций, а также сред разработки и тестирования, используемых проектной группой. Кроме того, системный администратор может участвовать в запуске разрабатываемой системы в промышленную эксплуатацию.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Тестер

Тестировщики проверяют, соответствует ли разрабатываемая система требованиям заказчика. Это может быть независимая команда, входящая в состав проектной группы.

Работа

специалистов по развертыванию связана с преобразованием существующих данных в формат, требуемый новой системой, и участием в развертывании новых версий.
Может быть независимой командой или частью проектной группы.

Технический писатель

Пользовательскую документацию готовят технические писатели, которые могут образовывать независимую команду или входить в состав проектной группы.

3.4.3. Практики

FDD состоит из набора «лучших практик», и разработчики метода утверждают, что, хотя выбранные практики не являются новыми, специфическое сочетание этих компонентов делает пять процессов FDD уникальными для каждого случая. Палмер и Фелсинг также считают, что для получения максимальной пользы от метода следует использовать все доступные практики, поскольку ни одна из них не доминирует над всем процессом (2002). FDD включает в себя следующие практики:

- Моделирование предметной области: исследование и объяснение предметной области проблемы.
Результатом становится структура, в которую добавляются необходимые функции.
- Разработка по функциям: разработка и отслеживание прогресса с помощью списка небольших функционально разложенных и ценных для клиента функций.
- Индивидуальное владение классом (кодом): для каждого класса назначается один человек, ответственный за согласованность, производительность и концептуальную целостность класса.
- Функциональные команды: относятся к небольшим, динамично формируемым командам.
- Инспекция: относится к использованию наиболее известных методов обнаружения дефектов механизмы.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

- Регулярные сборки: обеспечивают постоянную доступность работающей, демонстрируемой системы. Регулярные сборки формируют базовую версию, к которой добавляются новые функции.
- Управление конфигурацией: позволяет идентифицировать и отслеживать историю последних версий каждого завершённого файла исходного кода.
- Отчетность о ходе работ: отчетность о ходе работ предоставляется на основе завершённой работы на всех необходимых организационных уровнях.

Команда проекта должна применять все вышеперечисленные практики для соблюдения правил разработки FDD. Однако команда может адаптировать их в соответствии со своим опытом.

3.4.4. Принятие и опыт

Метод FDD был впервые использован в конце 90-х годов при разработке крупного и сложного банковского приложения. По мнению Палмера и Фелсинга (Palmer and Felsing, 2002), FDD подходит для новых проектов, проектов по улучшению и обновлению существующего кода, а также проектов, связанных с созданием второй версии существующего приложения. Авторы также рекомендуют организациям внедрять этот метод постепенно, небольшими частями, и, наконец, полностью, по мере развития проекта.

3.4.5. Область применения

Авторы далее утверждают, что метод FDD «заслуживает серьёзного рассмотрения любой организацией, занимающейся разработкой программного обеспечения, которой необходимо своевременно поставлять качественные, критически важные для бизнеса программные системы» (Palmer and Felsing, 2002, стр. xxiii). К сожалению, достоверные отчёты об опыте использования до сих пор трудно найти, хотя несколько консалтинговых компаний, занимающихся разработкой программного обеспечения, по всей видимости, поддерживают этот метод, что легко увидеть в Интернете.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.4.6 Текущие исследования

Поскольку достоверных примеров успешного применения FDD или, наоборот, сообщений о неудачах нет, любые исследования, обсуждения и сравнения такого рода представляли бы огромный интерес как для учёных, так и для практиков, позволяя им эффективнее применять FDD в каждом конкретном случае, решать, когда использовать FDD, а не какой-либо другой гибкий метод, а когда не использовать подход FDD. Однако такие исследования ещё предстоит провести. Поскольку метод относительно новый, он всё ещё развивается, и в будущем будет доступно всё больше вспомогательных инструментов.

3.5. Рациональный унифицированный процесс

Rational Unified Process (сокращенно RUP) был разработан Филиппом Крукхеном, Иваром Якобсеном и другими специалистами корпорации Rational в качестве дополнения к UML (Unified Modelling Language), стандартному в отрасли методу моделирования программного обеспечения.

RUP — это итеративный подход к объектно-ориентированным системам, который активно использует сценарии использования для моделирования требований и построения фундамента системы. RUP склоняется к объектно-ориентированной разработке. Он не исключает другие методы, хотя предлагаемый метод моделирования, UML, особенно подходит для объектно-ориентированной разработки (Jacobsen et al., 1994).

3.5.1 Процесс

Жизненный цикл RUP-проекта делится на четыре фазы: «Начало», «Проработка», «Конструирование» и «Переход» (см. рис. 10). Эти фазы делятся на итерации, каждая из которых направлена на создание демонстрируемого программного обеспечения. Продолжительность итерации может варьироваться от двух недель (и менее) до шести месяцев.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

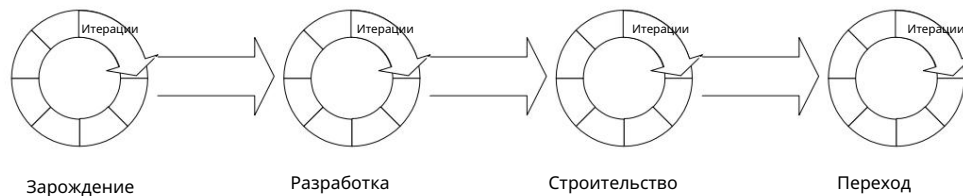


Рисунок 10. Фазы RUP

Далее вводятся фазы RUP, описанные в (Kruchten 2000):

На начальном этапе формулируются цели жизненного цикла проекта, учитывающие потребности всех заинтересованных сторон (например, конечного пользователя, покупателя или подрядчика). Это включает в себя определение, например, области применения, граничных условий и критериев приемки проекта. Выявляются критические варианты использования, то есть те, которые будут определять функциональность системы. Разрабатываются возможные архитектуры системы, а также оценивается график и стоимость всего проекта. Кроме того, составляются сметы для следующего этапа разработки.

На этапе разработки закладывается основа архитектуры программного обеспечения. Проблемная область анализируется с учетом всей создаваемой системы. На этом этапе определяется план проекта – если вообще принимается решение о переходе к следующим этапам. Чтобы принять это решение, RUP предполагает, что на этапе разработки будет создана достаточно прочная архитектура с достаточно стабильными требованиями и планами. Процесс, инфраструктура и среда разработки подробно описываются. Поскольку RUP делает акцент на автоматизации инструментов, на этапе разработки также создается ее поддержка. После этапа разработки определяется и описывается большинство вариантов использования и все действующие лица, описывается архитектура программного обеспечения и создается исполняемый прототип архитектуры. В конце этапа разработки проводится анализ для определения реализации рисков, стабильности видения будущего продукта, стабильности архитектуры и расхода ресурсов по сравнению с изначально запланированным.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

На этапе разработки все оставшиеся компоненты и функции приложения разрабатываются, интегрируются в продукт и тестируются. RUP рассматривает этап разработки как производственный процесс, в котором особое внимание уделяется управлению ресурсами, контролю затрат, сроков и качества. Результаты этапа разработки (альфа-, бета- и другие тестовые версии) создаются максимально оперативно, обеспечивая при этом надлежащее качество. На этапе разработки выполняется один или несколько релизов, прежде чем перейти к заключительному этапу перехода.

Переходная фаза наступает, когда программный продукт становится достаточно зрелым (определяется, например, путём мониторинга частоты запросов на изменение системы) для выпуска в сообщество пользователей. На основе отзывов пользователей выпускаются последующие версии для устранения нерешённых проблем или завершения отложенных функций. Переходная фаза включает бета-тестирование, пилотирование, обучение пользователей и специалистов по сопровождению системы, а также внедрение продукта в отделы маркетинга, дистрибуции и продаж. Часто проводится несколько итераций с выпуском бета-версий и общедоступных версий. Также разрабатывается пользовательская документация (руководства, учебные материалы).

На протяжении всех фаз параллельно реализуются девять рабочих процессов (Kruchten, 2000). Каждая итерация более или менее полно охватывает все девять рабочих процессов. К ним относятся: бизнес-моделирование, требования, анализ и проектирование, реализация, тестирование, управление конфигурацией и изменениями, управление проектами и управление окружающей средой. Большинство из них очевидны. Однако два из них реже встречаются в описаниях других процессов разработки ПО, поэтому далее они описаны более подробно.

Бизнес-моделирование используется для обеспечения удовлетворения бизнес-потребностей клиента. Анализ организационной структуры и бизнес-процессов клиента позволяет лучше понять структуру и динамику его бизнеса.

Бизнес-модель строится как модель бизнес-прецедентов, состоящая из бизнес-прецедентов и участников. Важность бизнес-моделирования полностью зависит от цели, для которой разрабатывается программное обеспечение, и этот этап может быть полностью пропущен, если в этом нет необходимости. Бизнес-моделирование чаще всего выполняется на этапах разработки и разработки.

Рабочий процесс «Окружающая среда» предназначен исключительно для поддержки разработки. Действия в этом рабочем процессе включают внедрение и настройку RUP.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Самостоятельно, выбор и приобретение необходимых инструментов, разработка собственных инструментов, обслуживание программного и аппаратного обеспечения, а также обучение. Практически все работы по технологическому процессу выполняются на начальном этапе.

3.5.2 Роли и обязанности

Назначение ролей в RUP основано на деятельности, то есть для каждой деятельности существует своя роль. RUP определяет тридцать ролей, называемых исполнителями. Состав ролей довольно стандартен (например, архитектор, проектировщик, рецензент проекта, менеджер по конфигурации), за исключением ролей, определённых в рабочих процессах бизнес-моделирования и среды.

В постоянном сотрудничестве с сотрудниками организации аналитик бизнес-процессов руководит и координирует определение бизнес-прецедентов, описывающих важные процессы и участников (например, клиентов) в бизнесе организации. Таким образом, он формирует высокоуровневую абстракцию моделируемого бизнеса в виде модели бизнес-прецедентов, а также определяет модель бизнес-объектов, описывающую взаимодействие процессов и участников, и создаёт глоссарий с перечнем важных терминов, используемых в бизнесе.

Модель бизнес-прецедентов делится на части, каждая из которых разрабатывается бизнес-проектировщиком в виде бизнес-прецедента. При этом он определяет и документирует роли и сущности внутри организации.

Рецензент бизнес-моделей просматривает все артефакты, созданные аналитиком бизнес-процессов и бизнес-дизайнером.

Рабочий процесс «Окружение» включает две интересные роли: разработчик курса создаёт учебные материалы (слайды, обучающие материалы, примеры и т. д.) для конечных пользователей разрабатываемой системы. Инструментальщик разрабатывает внутренние инструменты для поддержки разработки, повышения автоматизации трудоёмких повторяющихся задач и улучшения интеграции между инструментами.

Количество людей, необходимых для проекта RUP, значительно варьируется в зависимости от области применения рабочих процессов. Например,

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Рабочий процесс бизнес-моделирования (вместе с соответствующими ролями) можно полностью опустить, если бизнес-модель не имеет значения для конечного продукта.

3.5.3. Практики

Краеугольные камни RUP можно свести к шести практикам, лежащим в основе структуры фазового рабочего процесса и ролей RUP. Эти практики перечислены ниже.

Таблица 3. Практики RUP (Kruchten 2000).

практика RUP	Описание
Разрабатывайте программное обеспечение итеративно	Программное обеспечение разрабатывается небольшими порциями и короткими итерациями, что позволяет выявлять риски и проблемы на ранних стадиях и адекватно на них реагировать.
Управление требованиями	Определение требований к изменению системы программного обеспечения время и те требования, которые оказывают наибольшее влияние на Цели проекта – это непрерывный процесс. Дисциплинированный подход к требуется управление требованиями, где требования могут быть приоритизированы, отфильтрованы и отслежены. Несоответствия могут быть затем легче обнаружить. У коммуникации больше шансов успешны, когда основаны на четко определенных требованиях.
Использовать на основе компонентов архитектуры	Архитектуру программного обеспечения можно сделать более гибкой благодаря использованию компонентов: те части системы, которые подвержены наибольшему изменению, можно изолировать и упростить управление ими. Кроме того, создание повторно используемых компонентов может существенно сэкономить время на разработку в будущем.
Визуально моделировать программное обеспечение	Модели строятся, потому что сложные системы невозможно понять их в полном объеме. Используя общую визуализацию метод (например, UML) среди команды разработчиков, система Архитектура и дизайн могут быть поняты однозначно, и доведены до сведения всех заинтересованных сторон.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

практика RUP	Описание
Проверка качества программного обеспечения.	Благодаря тестированию на каждой итерации дефекты могут быть выявлены на более ранних этапах цикла разработки, что значительно снижает затраты на исправление их.
Изменения в управлении программным обеспечением	Любые изменения требований должны контролироваться, а их влияние Программное обеспечение должно быть отслеживаемым. Зрелость программного обеспечения также может быть эффективно измерена по частоте и типам внесены изменения.

3.5.4. Принятие и опыт

(Kruchten 2000) утверждает, что RUP часто можно внедрить полностью или частично «из коробки». Однако во многих случаях перед внедрением рекомендуется провести тщательное конфигурирование, то есть модификацию RUP. Конфигурирование представляет собой описание разработки, в котором перечислены все отклонения, которые необходимо внести в полную версию RUP.

Сам процесс внедрения представляет собой итеративную программу из шести шагов, которая повторяется до полного внедрения нового процесса, что, по сути, представляет собой цикл «планирование-выполнение-проверка-воздействие». Каждый этап включает в себя внедрение новых практик и корректировку ранее добавленных. При необходимости, на основе отзывов, полученных в ходе предыдущего цикла, сценарий разработки корректируется. Рекомендуется сначала провести пилотное тестирование процесса в рамках подходящего проекта.

Однако, несмотря на необходимость значительной адаптации, RUP был внедрён во многих организациях. Успех RUP может быть отчасти обусловлен тем, что Rational Software предлагает популярные инструменты для поддержки этапов RUP, например, Rational Rose, инструмент моделирования UML, и ClearCase, инструмент управления конфигурацией программного обеспечения.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.5.5 Область применения

RUP обычно не считается особенно «гибким». Изначально этот метод был разработан для всего процесса производства программного обеспечения и поэтому содержит подробные рекомендации по этапам процесса, которые практически неактуальны в среде, где, вероятно, потребуется гибкий подход. Однако недавние исследования (Martin, 1998; Smith, 2001) изучили потенциал RUP для использования этого метода в гибких условиях. По-видимому, этап внедрения является ключом к адаптации RUP к гибкости. Полный RUP содержит более сотни артефактов, которые должны быть созданы на различных этапах процесса, что требует тщательного отбора, в результате которого внедряются только самые необходимые.

Однако одним из главных недостатков RUP является отсутствие чётких рекомендаций по внедрению, подобных, например, Crystal, где перечислены необходимая документация и роли в зависимости от критичности программного обеспечения и размера проекта. RUP полностью оставляет адаптацию на усмотрение пользователя, что поднимает вопрос: какую часть RUP можно исключить, чтобы полученный процесс всё ещё оставался «RUP»?

3.5.6. Текущие исследования

dX Роберта Мартина — это минимальная версия RUP, в которой учтены принципы гибкой разработки программного обеспечения, что позволяет свести действия RUP к минимуму (Martin, 1998). Чтобы подчеркнуть гибкость RUP, dX намеренно копирует принципы XP, но в виде адаптированных действий RUP (помимо того, что «dX» означает «небольшое изменение», это перевёрнутое «XP»).

Гибкое моделирование (Ambler 2002a) — это новый подход (подробности см. в 3.9.1), который в RUP может использоваться для моделирования бизнеса, определения требований, а также на этапах анализа и проектирования.

3.6 Метод разработки динамических систем

С момента своего появления в 1994 году метод разработки динамических систем (DSDM) постепенно стал основой номер один для быстрого применения

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

(RAD) в Великобритании (Stapleton, 1997). DSDM — это некоммерческая и не имеющая патента платформа для разработки RAD, поддерживаемая Консорциумом DSDM10. Разработчики метода утверждают, что DSDM не только служит методом в общепринятом смысле, но и предоставляет платформу для контроля RAD, дополненную рекомендациями по эффективному использованию этих средств контроля.

Основная идея DSDM заключается в том, что вместо того, чтобы фиксировать объем функциональности продукта, а затем корректировать время и ресурсы для достижения этой функциональности, предпочтительнее фиксировать время и ресурсы, а затем соответствующим образом корректировать объем функциональности.

3.6.1 Процесс

DSDM состоит из пяти этапов: технико-экономическое обоснование, бизнес-исследование, итерация функциональной модели, итерация проектирования и разработки, а также внедрение (рис. 11). Первые два этапа являются последовательными и выполняются только один раз. Три последних этапа, в ходе которых выполняется фактическая разработка, являются итеративными и инкрементальными. В DSDM итерации рассматриваются как временные рамки. Временные рамки длятся в течение заранее определённого периода времени, и итерация должна завершиться в пределах этих рамок. Время, отведённое на каждую итерацию, и гарантированные результаты итерации планируются заранее. В DSDM типичная продолжительность временных рамок составляет от нескольких дней до нескольких недель.

¹⁰ См. www.dsdm.org.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

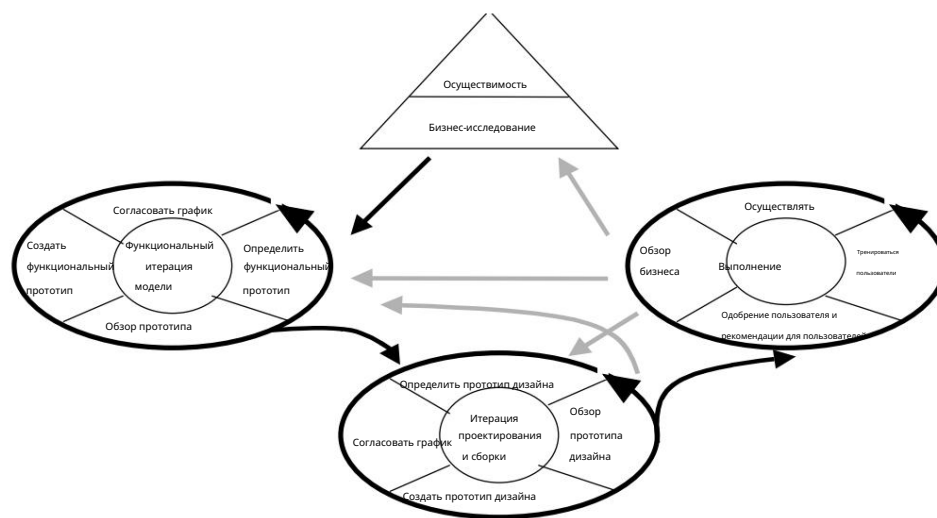


Рисунок 11. Диаграмма процесса DSDM (Stapleton 1997, стр. 3)

Далее будут представлены этапы и их основные выходные документы.

На этапе технико-экономического обоснования оценивается пригодность DSDM для конкретного проекта. Решение об использовании DSDM принимается с учётом типа проекта и, прежде всего, организационных и кадровых вопросов. Кроме того, на этапе технико-экономического обоснования рассматриваются технические возможности реализации проекта и связанные с ним риски. Подготавливаются два рабочих продукта: отчёт о технико-экономическом обосновании и план разработки.

При необходимости можно создать быстрый прототип, если бизнес или технология изучены недостаточно хорошо, чтобы решить, стоит ли переходить к следующему этапу. Ожидается, что этап технико-экономического обоснования займёт не более нескольких недель.

Бизнес-исследование – это этап, на котором анализируются основные характеристики бизнеса и технологий. Рекомендуемый подход заключается в организации семинаров с привлечением достаточного количества экспертов заказчика для рассмотрения всех важных аспектов системы и согласования приоритетов разработки. Затрагиваемые бизнес-процессы и категории пользователей описываются в определении бизнес-области. Определение затрагиваемых пользователей

занятия помогают вовлекать клиента, поскольку ключевые люди в жизни клиента

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Организация может быть распознана и вовлечена на раннем этапе. Общие описания процессов представлены в определении бизнес-области в удобном формате (ER-диаграммы, модели бизнес-объектов и т. д.).

На этапе бизнес-исследования формируются ещё два результата. Один из них — определение архитектуры системы, а другой — план прототипирования. Определение архитектуры — это первый набросок архитектуры системы, который может быть изменён в ходе проекта DSDM. План прототипирования должен содержать стратегию прототипирования для последующих этапов и план управления конфигурацией.

Фаза итерации функциональной модели – это первая итеративная и инкрементальная фаза. На каждой итерации планируются содержание и подход к итерации, итерация выполняется, а результаты анализируются для последующих итераций. Выполняются анализ и кодирование; создаются прототипы, а полученный опыт используется для улучшения аналитических моделей. Прототипы не следует полностью отбрасывать, а постепенно доводить до уровня качества, необходимого для включения в финальную систему. В результате создается функциональная модель, содержащая код прототипа и аналитические модели. Тестирование также является неотъемлемой частью этого этапа.

В фазе имеется еще четыре выхода (на разных этапах фазы).

Приоритетные функции – это список функций, которые будут реализованы в конце итерации. Документы по обзору функционального прототипирования содержат комментарии пользователей о текущем этапе, которые служат входными данными для последующих итераций. Перечисляются нефункциональные требования, в основном подлежащие рассмотрению на следующем этапе. Анализ рисков дальнейшей разработки – важный документ на этапе итерации функциональной модели, поскольку начиная со следующего этапа (итерации проектирования и сборки) возникающие проблемы будет сложнее решить.

На этапе проектирования и сборки система в основном разрабатывается. Результатом является протестированная система, соответствующая как минимум минимально согласованному набору требований. Проектирование и сборка являются итеративными, дизайн и функциональные прототипы проверяются пользователями, а дальнейшая разработка осуществляется на основе комментариев пользователей.

Заключительный этап внедрения — это перенос системы из среды разработки в реальную производственную среду. Обучение

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Система предоставляется пользователям, и система передается им. Если развертывание касается большого количества пользователей и выполняется в течение определенного периода времени, этап внедрения может быть итеративным. Помимо поставленной системы, результатом этапа внедрения также являются руководство пользователя и отчет об обзоре проекта. В отчете подводятся итоги проекта, и на его основе определяется направление дальнейшей разработки. DSDM определяет четыре возможных направления разработки. Если система соответствует всем требованиям, дальнейшая работа не требуется. С другой стороны, если необходимо оставить значительное количество требований (например, если они не были выявлены на этапе разработки системы), процесс может быть повторен от начала до конца. Если необходимо исключить некоторые менее важные функции, процесс может быть повторен, начиная с этапа итерации функциональной модели. Наконец, если некоторые технические проблемы невозможно решить из-за ограничений по времени, их можно решить путем повторного итерационного процесса, начиная с этапа итерации проектирования и сборки.

3.6.2 Роли и обязанности

DSDM определяет 15 ролей для пользователей и разработчиков. Наиболее распространенные из них, описанные в (Stapleton, 1997), перечислены ниже.

Разработчики и старшие разработчики — единственные роли, связанные с разработкой. Старшинство определяется опытом выполнения задач, которые выполняет разработчик. Должность старшего разработчика также указывает на уровень лидерства в команде. Роли разработчика и старшего разработчика охватывают всех сотрудников отдела разработки, будь то аналитики, дизайнеры, программисты или тестировщики.

Технический координатор определяет архитектуру системы и отвечает за техническое качество проекта. Он также отвечает за технический контроль проекта, например, за использование управления конфигурацией программного обеспечения.

Среди пользовательских ролей наиболее важной является роль пользователя-амбассадора. Соответствующие обязанности включают в себя распространение знаний сообщества пользователей о проекте и распространение информации о ходе проекта среди других пользователей. Это обеспечивает получение достаточного количества отзывов от пользователей. Пользователь-амбассадор должен быть представителем сообщества пользователей, которые в конечном итоге будут использовать систему. Поскольку пользователь-амбассадор вряд ли будет представлять все необходимые интересы,

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Однако с точки зрения пользователя определена дополнительная роль пользователя-консультанта. Пользователями-консультантами могут быть любые пользователи, представляющие важную точку зрения с точки зрения проекта. К ним относятся, например, ИТ-специалисты или финансовые аудиторы.

Визионер — это пользователь, обладающий наиболее точным пониманием бизнес-целей системы и проекта. Вероятно, именно он является инициатором идеи создания системы. Задача визионера — обеспечить раннее выявление основных требований и дальнейшее развитие проекта в правильном направлении с точки зрения этих требований.

Исполнительный спонсор — это представитель организации-пользователя, обладающий соответствующими финансовыми полномочиями и ответственностью. Таким образом, исполнительный спонсор имеет решающее значение в принятии решений.

3.6.3. Практики

Девять практик определяют идеологию и основу всей деятельности в DSDM. Эти практики, называемые принципами в DSDM, перечислены в таблице 4.

Таблица 4. Практики DSDM (Stapleton, 1997).

практика DSDM	Описание
Активное вовлечение пользователей является обязательным.	Несколько опытных пользователей должны присутствовать на всех этапах разработки системы, чтобы обеспечить своевременное и точное обратная связь.
Команды DSDM должны быть уполномоченный делать решения.	Длительные процессы принятия решений недопустимы в быстрых Циклы разработки. Пользователи, участвующие в разработке, имеют знания, позволяющие определить, куда должна двигаться система.
Основное внимание уделяется частой доставке продукции.	Ошибочные решения можно исправить, если цикл поставки короткий и пользователи могут предоставить точную обратную связь.
Фитнес для бизнеса	«Создайте правильный продукт, прежде чем создавать его правильно».

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

практика DSDM	Описание
цель является существенным критерием для принятия результаты.	основные бизнес-потребности системы удовлетворены, не следует стремиться к техническому совершенству в менее важных областях.
Итеративный и инкрементный развитие необходимо сходиться на точный бизнес решение.	Системные требования редко остаются неизменными с самого начала Проект до конца. Позволяя системам развиваться посредством итеративного разработки, ошибки могут быть обнаружены и исправлены на ранних стадиях.
Все изменения во время разработки обратимым.	В ходе развития легко может быть выбран неверный путь. Используя короткие итерации и гарантируя возможность возврата к предыдущим состояниям в процессе разработки, можно безопасно вернуться к неправильному пути, исправлено.
Требования: базовый уровень высокий.	Замораживание основных требований следует проводить только на высоком уровне. уровне, чтобы обеспечить возможность изменения подробных требований по мере необходимости. Это гарантирует, что основные требования будут учтены на ранняя стадия, но разработка может начаться до каждого Требование было заморожено. По мере развития, большую часть требований следует заморозить по мере их прояснения и согласованы.
Тестирование интегрировано на протяжении всего жизненного цикла.	В условиях жёстких временных ограничений тестирование, как правило, игнорируется, если его откладывают на конец проекта. Поэтому каждый компонент системы должен тестироваться разработчиками и пользователями по мере его разработки. Тестирование также является инкрементальным, и тесты становятся более комплексными на протяжении всего проекта. Регрессионное тестирование особенно важно в связи с эволюционным стилем разработки.
Совместный и кооперативный подход разделяемые всеми заинтересованными сторонами	Для того чтобы DSDM работала, организация должна взять на себя обязательство и его бизнес-отделы, а также ИТ-отделы взаимодействуют. Выбор то, что поставляется в системе и что остается вне ее, всегда является

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

практика DSDM	Описание
имеет важное значение.	компромисс и требует общего согласия. В меньшем масштабе обязанности системы распределены, поэтому сотрудничество пользователя и разработчика также должно осуществляться бесперебойно.

3.6.4. Принятие и опыт

Метод DSDM широко применяется в Великобритании с середины 90-х годов. Восемь клинических случаев описаны в работе (Stapleton, 1997), и этот опыт ясно показывает, что DSDM является жизнеспособной альтернативой RAD.

Для облегчения внедрения метода консорциум DSDM опубликовал фильтр пригодности метода, охватывающий три области: бизнес, системы и технические аспекты. Основные вопросы, которые необходимо рассмотреть, перечислены в таблице.

5.

Таблица 5. Соображения относительно усыновления (Стэйплтон, 1997, стр. 19-22).

Вопрос	Описание
Будет ли функциональность достаточно видима для пользователя интерфейса?	Если вовлечение пользователей осуществляется посредством создания прототипов, пользователи должны иметь возможность проверить, что программное обеспечение выполняет правильные действия без участия пользователя чтобы понять технические детали.
Можете ли вы четко определить все классы конечных пользователей?	Важно иметь возможность выявить и вовлечь все соответствующие группы пользователей программного продукта.
Является ли приложение вычислительно сложным?	Существует очевидная опасность в развитии сложных функциональности с нуля за счет использования DSDM. Лучшие результаты достигаются, если некоторые из Строительные блоки уже доступны для

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Вопрос	Описание
	команда разработчиков.
Является ли приложение потенциально большим? Если это так, можно ли его разделить на более мелкие части? функциональные компоненты?	DSDM использовался для разработки очень Большие системы. Некоторые из этих проектов имели Срок разработки составляет от 2 до 3 лет. Однако, функциональность должна быть разработана в достаточно дискретной форме строительные блоки.
Действительно ли проект ограничен по времени?	Активное вовлечение пользователей имеет первостепенное значение в методе DSDM. Если пользователи не участвуют в проекте разработки (поскольку проект фактически не ограничен по времени), разработчики могут разочароваться и начать делать предположения. о том, что необходимо для того, чтобы уложиться в сроки.
Являются ли требования гибкими и указано только на высоком уровне?	Если детальные требования были согласованы и исправлены до того, как разработчики программного обеспечения начнут свою работу, преимущества DSDM не будут достигнуты.

Эти и другие соображения по внедрению можно найти в руководстве DSDM (DSDMConsortium, 1997).

3.6.5 Область применения

Размер команды DSDM варьируется от двух до шести человек, и в проекте может быть задействовано несколько команд. Минимальное количество участников (два человека) обусловлено тем, что в каждой команде должен быть как минимум один пользователь и один разработчик. Максимальное количество участников (шесть) – это значение, найденное на практике. DSDM применяется как в небольших, так и в крупных проектах. Как уже упоминалось, обязательным условием для его использования в крупных системах является возможность разделения системы на компоненты, которые могут быть разработаны небольшими командами.

Рассматривая область применения, Стэплтон (1997) предполагает, что DSDM легче применять в бизнес-системах, чем в инженерных или научных приложениях.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.6.6 Текущие исследования

Метод DSDM изначально был разработан и продолжает поддерживаться консорциумом, состоящим из нескольких компаний-членов. Руководства по DSDM и сопутствующие документы предоставляются партнёрам консорциума за символическую ежегодную плату. За пределами консорциума не существует исследований, в то время как внутри консорциума метод продолжает развиваться. Например, в 2001 году была выпущена версия метода e-DSDM, адаптированная для электронного бизнеса и электронной коммерции. проекты были выпущены (Хайсмит 2002).

3.7. Разработка адаптивного программного обеспечения

Адаптивная разработка программного обеспечения (ASD) была разработана Джеймсом А. Хайсмитом III, опубликовано в (Highsmith, 2000). Многие принципы ASD основаны на более ранних исследованиях Хайсмита по методам итеративной разработки. Наиболее примечательный предшественник ASD, «RADical Software Development», был написан Хайсмитом в соавторстве с С. Байером и представлен в (Bayer and Highsmith, 1994).

ASD фокусируется главным образом на проблемах разработки сложных, крупных систем. Метод настоятельно рекомендует поэтапную, итеративную разработку с постоянным созданием прототипов. По сути, ASD — это «балансирование на грани хаоса»: его цель — предоставить структуру с достаточными ориентирами, чтобы предотвратить скатывание проектов в хаос, но не слишком сильными, чтобы не подавлять новаторство и креативность.

3.7.1 Процесс

Проект разработки адаптивного программного обеспечения реализуется в трехфазных циклах. Фазы циклов — «Размышление», «Сотрудничество» и «Обучение» (см. рисунок 12).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

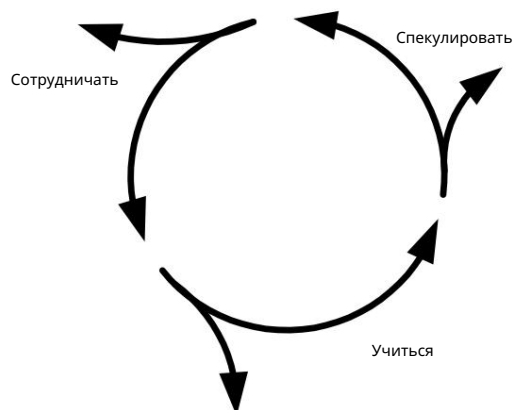


Рисунок 12. Цикл РАС (Хайсмит 2000, стр. 18).

Названия фаз призваны подчеркнуть роль изменений в этом процессе.

Вместо «планирования» используется слово «размышление», поскольку «план» обычно рассматривается как нечто, где неопределённость является слабостью, а отклонения от него указывают на провал. Аналогично, «сотрудничество» подчёркивает важность командной работы как средства разработки систем с высокой степенью изменений. «обучение» подчёркивает необходимость признавать ошибки и реагировать на них, а также тот факт, что требования могут меняться в ходе разработки.

На рисунке 13 циклы адаптивной разработки представлены более подробно. На этапе инициации проекта определяются его краеугольные камни, и он начинается с формулирования миссии проекта. Миссия, по сути, задаёт общие рамки конечного продукта, и вся разработка направляется на её выполнение.

Один из важнейших моментов при определении миссии проекта — определить, какая информация необходима для его реализации. Важные аспекты миссии определяются тремя пунктами: уставом видения проекта, паспортом проекта и описанием спецификации продукта. Значение этих документов подробно объясняется в работе (Highsmith, 2000). На этапе инициации проекта определяется общий график, а также графики и цели циклов разработки. Циклы обычно делятся от четырёх до восьми недель.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

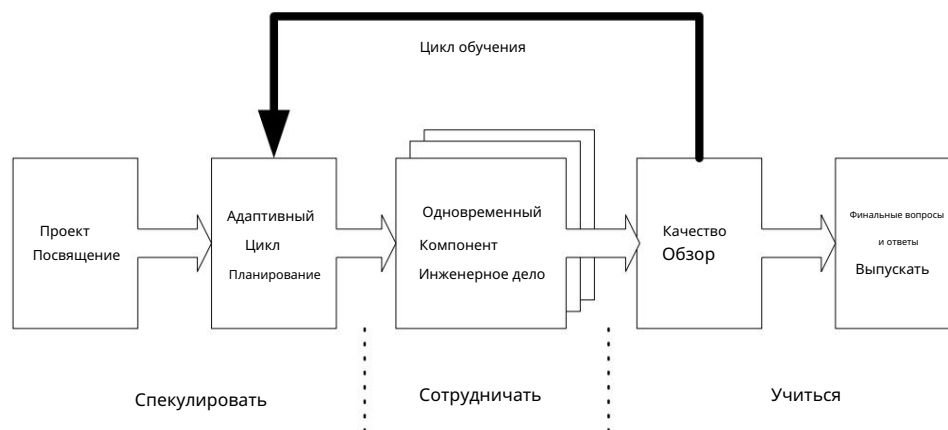


Рисунок 13. Фазы жизненного цикла детей с РАС (Хайсмит 2000, стр. 84).

ASD явно ориентирован на компоненты, а не на задачи. На практике это означает, что основное внимание уделяется результатам и их качеству, а не задачам или процессу, используемому для их получения. ASD решает эту проблему посредством адаптивных циклов разработки, включающих фазу совместной работы, где несколько компонентов могут разрабатываться одновременно. Планирование циклов является частью итеративного процесса, поскольку определения компонентов постоянно уточняются с учетом новой информации и изменений требований к компонентам.

Основой для последующих циклов («Петля обучения» на рисунке 13) служат повторные обзоры качества, направленные на демонстрацию функциональности программного обеспечения, разработанного в ходе цикла. Важным фактором при проведении обзоров является присутствие заказчика в составе группы экспертов (так называемой клиентской фокус-группы). Однако, поскольку обзоры качества проводятся довольно редко (только в конце каждого цикла), присутствие заказчика в ASD подкрепляется сессиями совместной разработки приложений (JAD). Сессия JAD по сути представляет собой семинар, на котором разработчики и представители заказчика встречаются для обсуждения желаемых функций продукта и улучшения коммуникации. ASD не предлагает графиков проведения сессий JAD, но отмечает их особую важность в начале проекта.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Заключительный этап проекта разработки адаптивного программного обеспечения — финальные вопросы и ответы. и этап выпуска. ASD не рассматривает, как следует проводить эту фазу, но подчеркивает важность извлечения уроков из пройденного материала. Посмертные материалы проекта рассматриваются как критически важные в высокоскоростных и быстро меняющихся проектах, где происходит гибкая разработка.

Подводя итог, можно сказать, что адаптивный характер развития при PAC характеризуется: следующие свойства:

Таблица 6. Характеристики циклов адаптивного развития (Хайсмит 2000).

Характеристика	Описание
Движимый миссией	Действия в каждом цикле разработки должны быть обоснованы Общая миссия проекта. Миссия может быть скорректирована по мере необходимости. разработка продолжается.
Компонентно-ориентированный	Деятельность по развитию не должна быть ориентирована на выполнение задач, а скорее на сосредоточиться на разработке работающего программного обеспечения, т.е. на построении системы понемногу за раз.
Итеративный	Метод последовательного водопада работает только в хорошо понятных и чётко определённых средах. Развитие в большинстве случаев идёт бурно, и Поэтому усилия по развитию должны быть сосредоточены на переделке вместо того, чтобы сделать все правильно с первого раза.
Ограниченные по времени	Неопределенность в сложных программных проектах может быть устранена путем Регулярно устанавливая конкретные сроки. Проект с фиксированными сроками. руководство заставляет участников проекта принимать неизбежные и трудные компромиссные решения на ранних этапах проекта.
Толерантный к изменениям	Изменения в разработке программного обеспечения происходят часто. Поэтому важнее уметь к ним приспосабливаться, чем контролировать Чтобы создать систему, устойчивую к изменениям, разработчики должны постоянно проверять, являются ли компоненты, которые они строят, вероятно, изменится.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Характеристика	Описание
Ориентированный на риск	Разработка элементов с высокой степенью риска (например, наименее известных или наиболее важных в случае внесения изменений) должна начинаться как можно раньше.

3.7.2 Роли и обязанности

Процесс ASD во многом берет начало в организационной и управленческой культуре, и особенно в важности взаимодействия команд и командной работы. Все эти вопросы подробно рассматриваются в работе (Highsmith, 2000). Однако в этом подходе не описывается подробно структура команд. Также перечислено очень мало ролей и обязанностей. «Исполнительный спонсор» назван лицом, несущим общую ответственность за разрабатываемый продукт. Единственными другими упомянутыми ролями являются участники совместного сеанса разработки приложения (фасилитатор, планирующий и ведущий сеанс, секретарь, ведущий протокол, менеджер проекта, а также представители заказчика и разработчика).

3.7.3. Практики

ASD предлагает очень мало практик для повседневной работы по разработке программного обеспечения. По сути, (Хайсмит, 2002) четко называет три: итеративную разработку, планирование на основе функций (компонентов) и обзоры фокус-групп клиентов. Действительно, пожалуй, самая серьезная проблема, связанная с РАС, заключается в том, что её практики трудно идентифицировать, и многие детали остаются открытыми. В РАС практически нет практик, которые стоило бы применять; большинство вопросов, рассматриваемых в книге, скорее являются примерами того, что можно было бы сделать.

3.7.4. Принятие и опыт

Принципы и лежащие в основе ASD идеи разумны, но практических рекомендаций по применению этого метода мало. Книга (Highsmith, 2000) предлагает ряд идей по разработке программного обеспечения в целом и предлагает

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Очень интересное чтение. Основная философия книги — создание адаптивной организационной культуры, а не её конкретика. В результате метод не даёт практически никакой основы для внедрения аутизма в организации.

Конечно, наряду с этим необходимо использовать и другие методы. Однако это не означает, что ASD бесполезно. Напротив, ASD, несомненно, закладывает важную философию, лежащую в основе гибкого подхода, предлагая идеи разработки сложных систем, сотрудничества и гибкости в культуре управления.

3.7.5 Область применения

Методология ASD не имеет встроенных ограничений в применении. Интересной особенностью применения ASD является то, что она не требует использования совместно работающих команд, как большинство других гибких методов разработки программного обеспечения. Разработка сложных систем, как известно, требует обширных знаний множества технологий, что делает необходимым объединение опыта множества специализированных команд.

Зачастую командам приходится работать вне пространственных, временных и организационных границ. Хайсмит (2000) отмечает это и указывает, что большинство трудностей в распределенной разработке связано с социальными, культурными и командными навыками. Поэтому ASD предлагает методы для улучшения межкомандного взаимодействия, предлагая стратегии обмена информацией, использование инструментов коммуникации и способы постепенного повышения строгости проектной работы для поддержки распределенной разработки.

3.7.6 Текущие исследования

Существенных исследований ASD не проводилось. Возможно, из-за того, что метод допускает широкие возможности для адаптации, практически нет сообщений об опыте использования ASD как такового. Однако основополагающие принципы адаптивной разработки программного обеспечения не теряют своей актуальности. Создатель ASD развивает эту тему дальше, уделяя особое внимание ключевым составляющим сред гибкой разработки программного обеспечения, а именно людям, отношениям и неопределенности. Часть этой работы он изложил в недавно вышедшей книге «Экосистемы гибкой разработки программного обеспечения» (Highsmith, 2002).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.8 Разработка программного обеспечения с открытым исходным кодом

В этой главе обсуждается разработка программного обеспечения с открытым исходным кодом (OSS). Как отмечалось в предыдущей главе, она имеет ряд сходств с другими (гибкими) методами разработки программного обеспечения, хотя и имеет свои особенности. Сочетание изобретения электронных досок объявлений и давней традиции разработчиков программного обеспечения свободно обмениваться кодом между коллегами усилилось и стало возможным в глобальном масштабе с развитием Интернета в 90-х годах. Этот процесс разработки вдохновил на создание новой парадигмы разработки программного обеспечения с открытым исходным кодом (OSS), предлагающей инновационный способ создания приложений. Это вызвало растущий интерес, а также многочисленные исследования и дискуссии, касающиеся возможностей и значения парадигмы OSS для всей индустрии разработки программного обеспечения. Этот интерес значительно возрос после нескольких успешных проектов, среди которых сервер Apache, язык программирования Perl, почтовый обработчик SendMail и операционная система Linux. Microsoft даже назвала последнюю своим самым серьезным конкурентом на рынке серверных операционных систем.

Парадигма открытого исходного кода предполагает свободный доступ к исходному коду для модификации и распространения без каких-либо затрат. Парадигму открытого исходного кода можно также рассматривать с философской точки зрения (O'Reilly, 1999; Hightower и Lesiecki, 2002). Феллер и Фицджеральд (2000) выделяют следующие мотивы и движущие силы разработки открытого исходного кода:

- 1) Технологические: потребность в надежном коде, более быстрых циклах разработки, более высоких стандартах качества, надежности и стабильности, а также в более открытых стандартах и платформах.
- 2) Экономичность; корпоративная потребность в распределении затрат и рисков
- 3) Социально-политические: удовлетворение «личных потребностей» разработчика, репутация среди коллег, стремление к «значимой» работе и идеализм, ориентированный на сообщество.

Большинство известных проектов разработки ПО с открытым исходным кодом сосредоточены на инструментах разработки или других платформах, которые используются профессионалами, часто участвующими в

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

сами усилия по разработке, таким образом, выполняя одновременно функции заказчика и разработчика. Проект ПО с открытым исходным кодом (OSS) не является сборником чётко определённых и опубликованных практик разработки программного обеспечения, представляющих собой красноречивый письменный метод. Вместо этого его лучше описать в терминах различных лицензий на распространение программного обеспечения и как совместный способ создания программного обеспечения разбросанными по всему миру людьми небольшими и частыми инкрементами. Инициатива открытого исходного кода (Open Source Initiative)¹¹ отслеживает и предоставляет лицензии на программное обеспечение, соответствующее определениям OSS. Некоторые исследователи, например (Gallivan, 2001; Crowston and Scozzi, 2002), предполагают, что проект OSS фактически представляет собой виртуальную организацию.

3.8.1 Процесс

Хотя Кокберн (2002a) отмечает, что разработка ПО с открытым исходным кодом отличается от метода гибкой разработки в философском, экономическом и командном аспектах, во многом разработка ПО с открытым исходным кодом следует тем же принципам и практикам, что и другие гибкие методы. Например, процесс разработки ПО с открытым исходным кодом начинается с ранних и частых релизов, и в нём отсутствуют многие традиционные механизмы координации разработки ПО с помощью планов, системных проектов, графиков и определённых процессов. Как правило, проект ПО с открытым исходным кодом состоит из следующих видимых фаз (Шарма и др., 2002):

1. Обнаружение проблемы
2. Поиск волонтеров
3. Идентификация решения
4. Разработка и тестирование кода
5. Проверка изменений кода
6. Фиксация кода и документирование

¹¹ www.opensource.org, (13.5.2002)

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

7. Управление релизами

Несмотря на то, что методы разработки программного обеспечения с открытым исходным кодом можно описать с помощью вышеуказанных этапов итерации, интерес представляет то, как этот процесс управляется. Можно увидеть, как метод разработки программного обеспечения с открытым исходным кодом охарактеризован Мокусом и др. (2000) следующими утверждениями:

- Системы создаются потенциально большим количеством добровольцев.
- Работа не назначается; люди сами выбирают себе задачу увлекающийся
- Не существует явного дизайна уровня системы.
- Отсутствует план проекта, график или список результатов.
- Система расширяется небольшими порциями
- Программы часто тестируются

Согласно Феллеру и Фицджеральду (2000), процесс разработки ПО с открытым исходным кодом организован как масштабная параллельная разработка и отладка. Это включает в себя слабо централизованный, кооперативный и бесплатный вклад отдельных разработчиков. Однако есть признаки того, что эта «бесплатная» концепция разработки меняется. Процесс разработки ПО с открытым исходным кодом не предполагает каких-либо predetermined формальных норм документирования или традиций. Однако этот процесс включает в себя традиции и табу, которые усваиваются и осознаются с опытом.

3.8.2 Роли и обязанности

Процесс разработки программного обеспечения с открытым исходным кодом (OSS) кажется довольно свободным и непредсказуемым. Однако он должен иметь определённую структуру, иначе он никогда не смог бы достичь столь впечатляющих результатов, как в последние годы. Кроме того, интерес к OSS начали проявлять и старые, солидные компании, среди которых особенно выделяются IBM, Apple, Oracle, Corel, Netscape, Intel и Ericsson (Феллер и Фицджеральд, 2000). В этих усилиях

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

особенно в крупных проектах OSS, названные компании взяли на себя роль координатора и посредника, выступая, таким образом, в качестве управляющих проектами.

Другим новым явлением стало появление сайтов-аукционов или репозиторий, таких как Freshmeat¹² и SourceForge¹³, которые создали сайты разработки с обширным репозиторием открытого исходного кода и приложений, доступных в Интернете. Эти сайты обычно предоставляют разработчикам открытого исходного кода бесплатные услуги, включая хостинг проектов, контроль версий, отслеживание ошибок и проблем, управление проектами, резервное копирование и архивирование, а также ресурсы для общения и совместной работы. Поскольку в контексте открытого исходного кода не используются личные встречи, важность Интернета очевидна. Процесс разработки неизбежно нуждается в хорошо функционирующем программном обеспечении для управления версиями, которое строго отслеживает и позволяет отправлять и оперативно тестировать новый исходный код. Роль этих инструментов нельзя недооценивать, поскольку разрабатываемое программное обеспечение должно быть организовано и работать непрерывно, днем и ночью, а сами разработчики имеют очень разный уровень навыков и опыта.

По мнению Галливана (2001), типичная структура разработки СПО состоит из нескольких уровней добровольцев:

- 1) Руководители проектов, которые несут общую ответственность за проект и которые обычно писали исходный код
- 2) Разработчики-волонтеры, которые создают и отправляют код для проекта.
может быть дополнительно определен как:
 - a. Старшие члены или основные разработчики с более широкими общими властью
 - b. Разработчики периферийных устройств, вносящие и отправляющие изменения в код.
 - c. Случайные авторы

¹² freashmeat.net, 20.05.2002

¹³ sourceforge.net, 20.05.2002

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

г. Сопровождающие и авторизованные разработчики

3) Другие лица, которые в ходе использования программного обеспечения проводят тестирование, выявляют ошибки и предоставляют отчеты о проблемах с программным обеспечением.

4) Авторы постов часто участвуют в группах новостей и обсуждениях, но не делают этого.
кодирование.

Шарма и др. (2002) утверждают, что проекты разработки ПО с открытым исходным кодом обычно делятся главными архитекторами или дизайнерами на более мелкие и легко выполнимые задачи, которые затем выполняются отдельными лицами или группами. Разработчики-волонтеры делятся на индивидуальных или небольших групп. Они свободно выбирают задачи, которые хотят выполнить. Таким образом, рациональное модульное разделение всего проекта имеет решающее значение для успешного завершения процесса разработки. Более того, эти подзадачи должны быть интересными, чтобы привлечь разработчиков.

3.8.3.Практики

Начать или приобрести право собственности на проект OSS можно несколькими способами: основать новый проект, передать его бывшему владельцу или добровольно взять на себя управление текущим умирающим проектом (Bergquist and Ljungberg 2001).

Зачастую будущие пользователи сами определяют продукт (проект) и сами занимаются кодированием.

Этот процесс непрерывен, поскольку разработка программного обеспечения постоянно развивается.

Несмотря на то, что в проектах разработки ПО с открытым исходным кодом могут участвовать сотни добровольцев, обычно основную часть работы выполняет лишь небольшая группа разработчиков (Ghosh and Prakash 2000; Mockus et al.).

2000; Демпси и др. 2002).

Шарма и другие (2002) описывают некоторые центральные организационные аспекты подхода «открытое программное обеспечение», например, разделение труда, механизм координации, распределение полномочий по принятию решений, организационные границы и неформальную структуру проекта. Мокус и другие (2000) характеризуют разработку «открытого программного обеспечения» следующим образом:

- Системы OSS создаются потенциально большим числом добровольцев.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

- Работа не назначается; люди выполняют ту работу, которую сами выбирают предпринять
- Отсутствует явный проект на системном уровне или даже детальный проект.
- Отсутствует план проекта, график или список результатов.

Для успешной работы географически разбросанные отдельные лица, а также небольшие группы разработчиков должны иметь хорошо функционирующие и открытые каналы связи между собой, тем более что разработчики обычно не встречаются лицом к лицу.

3.8.4. Принятие и опыт

Лернер и Тироль (2001) вкратце описывают некоторые аспекты разработки программного обеспечения Linux, Perl и Sendmail, описывая разработку этих приложений от усилий одного программиста до целой глобальной группы людей, численность которой может исчисляться десятками тысяч.

Одна из проблем процесса разработки ПО с открытым исходным кодом (OSS) и одновременно одно из его преимуществ заключается в том, что требования к ним постоянно совершенствуются, и продукт никогда не достигает финальной стадии, пока не будут завершены все работы по его разработке (Феллер и Фицджеральд, 2000). Метод разработки ПО с открытым исходным кодом (OSS) в значительной степени зависит от Интернета, обеспечивающего глобальное сетевое взаимодействие.

Ещё один важный вопрос, связанный с программным обеспечением с открытым исходным кодом, — это стабилизация и коммерциализация кода до уровня приложения. Поскольку сам код с открытым исходным кодом не является коммерческим продуктом, появился совершенно новый вид сервисного бизнеса, ориентированный на создание коммерчески успешных продуктов на основе этого открытого кода. Эти новые сервисные компании взяли на себя задачу создания готовых приложений со всеми необходимыми для коммерческой жизнеспособности функциями. Примерами таких систем являются, например, Redhat и SOT для Linux. Программное обеспечение, разработанное в рамках парадигмы открытого программного обеспечения (OSS), также может использоваться в новых бизнес-проектах, в этом случае вся бизнес-идея основана на использовании исключительно программного обеспечения с открытым исходным кодом (Wall, 2001).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3.8.5 Область применения

В настоящее время наиболее широко публикуемые результаты сосредоточены преимущественно на разработке крупных программных инструментов и интернет-продуктов (Lerner and Tirole, 2001). Сам метод разработки ПО с открытым исходным кодом не предполагает каких-либо специальных областей применения и не устанавливает ограничений на размер программного обеспечения, при условии, что предлагаемые проекты разработки соответствуют элементам, на которых основаны методы разработки ПО с открытым исходным кодом, описанные выше.

Будущие тенденции также зависят от использования преимуществ глобального сотрудничества разработчиков, в то время как разработка ПО с открытым исходным кодом постоянно генерирует сырьё, которое впоследствии может быть использовано в коммерческих целях (O'Reilly, 1999). С юридической точки зрения парадигму разработки ПО с открытым исходным кодом следует рассматривать скорее как структуру лицензирования, эксплуатирующую условия Стандартной общественной лицензии. Типичными примерами приложений с открытым исходным кодом, по мнению Феллера (2000), являются сложные системы поддержки инфраструктуры бэк-офиса с большим количеством пользователей.

Сам процесс разработки ПО с открытым исходным кодом может быть реализован с использованием различных методов разработки ПО, которые должны соответствовать и учитывать особенности парадигмы ПО с открытым исходным кодом. Таким образом, процесс может обладать гибкими аспектами, но, с другой стороны, сам процесс разработки может замедляться по ряду причин, например, из-за нехватки заинтересованных и опытных разработчиков, а также из-за противоречивых мнений.

Организации, разбросанные по территории и культуре, могли бы извлечь пользу из анализа плюсов и минусов различных методов открытого ПО и использования лучших из них в своем конкретном контексте разработки программного обеспечения.

3.8.6 Текущие исследования

Ведётся множество исследований, чтобы объяснить противоречивость успеха проектов с открытым исходным кодом. Как это работает, если формального метода разработки программного обеспечения с открытым исходным кодом не существует, но группа разработчиков, как правило, способна создавать хорошо функционирующее программное обеспечение? Одной из актуальных тем являются юридические аспекты, связанные с использованием компонентов с открытым исходным кодом. Феллер и Фицджеральд (2000) проанализировали практику лицензирования программного обеспечения с открытым исходным кодом, а Лернер и Тироль (2001) кратко рассматривают вопрос оптимального лицензирования в среде программного обеспечения с открытым исходным кодом. Они обсуждают проблемы «безбилетника» и

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

«перехват» кода, разработанного в ходе совместных действий, для дальнейшего коммерческого использования.

Компании-разработчики программного обеспечения проявляют все больший интерес к изучению возможностей использования методологии OSS для поддержки своей повседневной работы. Например, Шарма и другие (2002) исследовали эту проблему и предложили структуру для создания гибридной структуры сообщества разработчиков ПО с открытым исходным кодом. Различные компании также изучают неизбежные последствия новых бизнес-моделей, связанных с методами производства и распространения ПО с открытым исходным кодом (O'Reilly, 1999). Тем не менее, по словам Шармы и других (2002), было проведено лишь несколько эмпирических исследований, а строгих исследований того, как традиционные организации-разработчики ПО могут внедрять практики ПО с открытым исходным кодом и извлекать из них выгоду, практически не проводилось.

Сетевой эффект имеет решающее значение для успеха разработки ПО с открытым исходным кодом. Новые прототипы появляются в сети. Однако сообщество сурово, оно очень похоже на обычный деловой мир: если вы находите клиентов, вы выживаете, но без клиентов вы умираете. То есть, если представленный прототип привлекает достаточный интерес, то он постепенно начинает привлекать всё больше и больше разработчиков. В некоторых случаях ПО с открытым исходным кодом предоставляет сырьё, которое можно дорабатывать и расширять для удовлетворения коммерческих потребностей. Бергквист и Юнгберг (Bergquist and Ljungberg 2001, p. 319) изучали развитие отношений в сообществах ПО с открытым исходным кодом. Они приходят к выводу, что «эту культуру [ПО с открытым исходным кодом] можно понимать как своего рода слияние коллективизма и индивидуализма: именно отдача сообществу делает человека героем в глазах других. Герои оказывают важное влияние, но также являются сильными лидерами».

Кроустон и Скоцци (2002) исследовали несколько проектов открытого ПО, используя данные, доступные на Sourceforge.net. Их основные положения таковы:

1. Наличие компетенций является ключевым фактором успеха проекты.
2. Разработчикам важно уметь понимать потребности заказчиков проекта. Кроме того, проекты, связанные с разработкой и администрированием систем, оказались более успешными.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

3. Проекты, в которых участвуют известные администраторы, более успешны, так как им легче привлекать новых волонтеров.

3.9 Другие гибкие методы

В предыдущих подразделах было представлено несколько методов создания программного обеспечения по гибкой методологии. Основное внимание уделялось методам, которые предоставляют комплексные фреймворки с более или менее ощутимыми процессами и практиками, охватывающими максимально возможную часть процесса разработки программного обеспечения.

Однако в последнее время гибкий подход породил множество исследований и интересных новых идей, которые либо не были подробно документированы (например, бережливая разработка программного обеспечения¹⁴), либо были опубликованы совсем недавно (например, гибкое моделирование). Именно эти методы будут рассмотрены в данном разделе, поскольку они, как можно утверждать, предлагают интересные точки зрения в различных областях гибкой разработки.

3.9.1 Гибкое моделирование

Гибкое моделирование (Ambler, 2002a) – это новый подход к моделированию. В основе АМ лежат практики и культурные принципы. Основная идея заключается в том, чтобы стимулировать разработчиков создавать достаточно продвинутые модели для решения актуальных задач проектирования и документирования, при этом сохраняя минимальное количество моделей и документации. Основное внимание к культурным вопросам находит отражение в поддержке коммуникации, структуре команд и способах совместной работы команд.

Четыре ценности, лежащие в основе Agile Modeling, те же, что и в экстремальном программировании: коммуникация, простота, обратная связь и смелость.

По сути, они продвигают те же проблемы, что и в XP. Кроме того, Амблер упоминает смирение, подчёркивая тот факт, что у разных людей разные типы

¹⁴ www.poppendieck.com/

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

экспертные знания, которые следует использовать путем поощрения сотрудничества и привлечения лучших людей для выполнения данной работы.

Основные принципы Agile Modeling подчеркивают важность функционирования программного обеспечения как главной цели моделирования. Как и в случае с XP, эти принципы отчасти проистекают из суровой реальности: изменения в разработке ПО происходят часто и должны учитываться при моделировании. Основные принципы подкрепляются дополнительными принципами, помогающими создавать эффективные модели.

Суть Agile Modeling заключается в его практиках. Автор отмечает, что для того, чтобы моделирование было Agile Modeling, необходимо соблюдать каждый из основных принципов, однако практики, перечисленные в АМ, могут применяться независимо друг от друга. Agile Modeling в значительной степени ориентирован на практики. Одиннадцать лучших практик перечислены в качестве основных в четырёх категориях (Итеративное и инкрементальное моделирование, Командная работа, Простота, Валидация). Эти практики варьируются от выбора подходящего метода моделирования для моделируемого объекта до преимуществ командной работы. Восемь дополнительных практик также перечислены в трёх категориях (производительность, документирование, мотивация).

Agile Modeling — это не просто набор практик, но и глубокое понимание того, как эти практики сочетаются друг с другом и как они должны подкрепляться аспектами организационной культуры, повседневной рабочей среды, инструментами и командной работой. Как и многие гибкие методы, Agile Modeling отдаёт предпочтение взаимодействию с клиентами, работе в небольших командах и тесному взаимодействию. Однако нет ограничений на размер систем, которые можно построить, используя Agile Modeling в качестве принципа моделирования.

С более широкой точки зрения разработки программного обеспечения, Agile Modeling само по себе недостаточно. Для него требуются вспомогательные методы, поскольку он охватывает только моделирование, но даже это рассмотрение намеренно ограничено, так что в книге не описывается, например, создание UML-моделей. Однако, поскольку цель Agile Modeling — поддержка повседневной разработки программного обеспечения, значительные усилия были направлены на демонстрацию возможностей бесшовной интеграции Agile Modeling с другими методологиями разработки. В качестве примеров комбинирования Agile Modeling с другими методологиями разработки представлены Extreme Programming и Rational Unified Process.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Гибкое моделирование успешно демонстрирует, что многие принципы гибкой разработки программного обеспечения одинаково хорошо подходят для моделирования, и предоставляет продуманные иллюстрации того, как принципы гибкой разработки влияют на моделирование. Однако гибкое моделирование — это новый метод, и пока нет значимых исследований, посвящённых ему.

3.9.2 Прагматическое программирование

Название книги несколько вводит в заблуждение, поскольку метода под названием «прагматическое программирование» не существует. Вместо него существует интересный набор рекомендаций по программированию, опубликованный в книге «The Pragmatic Programmer» Эндрю Ханта и Дэвида Томаса (2000). Здесь, для простоты, этот подход будет называться «прагматическим программированием».

Причина включения прагматического программирования в этот обзор заключается в том, что оба автора книги участвовали в движении Agile и были одними из создателей Agile Manifesto. Что ещё важнее, описанные в книге методы весьма конкретно дополняют практики, обсуждаемые в других методах.

В прагматичном программировании нет процесса, фаз, чётко выраженных ролей или рабочих продуктов. Тем не менее, оно охватывает большинство практических аспектов программирования весьма убедительно и обоснованно. На протяжении всей книги авторы излагают основные положения в виде кратких советов. Многие из советов (всего их 70) сосредоточены на решении повседневных задач, но философия, лежащая в их основе, достаточно универсальна и может быть применена к любому этапу разработки программного обеспечения.

Философия определяется шестью пунктами, которые можно сформулировать примерно так:

- Берите на себя ответственность за свои действия. Думайте о решениях, а не об оправданиях.
- Не миритесь с плохим дизайном или кодом. Исправляйте несоответствия по мере их появления или планируйте их исправление в ближайшее время.
- Принимайте активное участие в реализации изменений там, где вы считаете это необходимым.
- Создавайте программное обеспечение, которое удовлетворяет ваших клиентов, но знайте меру.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

- Постоянно расширяйте свои знания.

- Улучшайте свои коммуникативные навыки.

С точки зрения Agile, наиболее интересные практики фокусируются на инкрементальной, итеративной разработке, тщательном тестировании и проектировании, ориентированном на пользователя. Этот подход имеет практическую направленность, и большинство практик иллюстрируются положительными и отрицательными примерами, часто дополненными вопросами и фрагментами кода. Значительные усилия направлены на объяснение того, как проектировать и внедрять программное обеспечение, чтобы оно выдерживало изменения. Также обсуждаются решения для поддержания согласованности программного обеспечения при любых изменениях, одним из которых является рефакторинг.

Подход прагматического программирования к тестированию предполагает реализацию тестового кода параллельно с реальным кодом и автоматизацию всех тестов. Идея заключается в том, что если не добавлять каждую исправленную ошибку в тестовую библиотеку и не проводить регрессионные тесты регулярно, время и усилия будут тратиться на повторный поиск одних и тех же ошибок, а негативные последствия изменений в коде не будут обнаружены достаточно рано.

В прагматичном программировании автоматизация также приветствуется для многих других задач. Более того, совет № 61 в книге доходит до того, что предлагает «не использовать ручные процедуры». Например, создание черновиков документации из исходного кода, создание кода из определений базы данных и автоматизация ежедневных сборок упоминаются как задачи, которые следует автоматизировать. Приводятся примеры подходящего программного обеспечения для внедрения автоматизации.

Книга, занимающая чуть меньше страниц, чем описанные выше практики, рассматривает важность активного поиска требований и поддержания разумного уровня детализации спецификаций. Целью качества на этих этапах, как и на протяжении всей книги, является предоставление «достаточно хорошего программного обеспечения».

Вкратце, прагматичное программирование демонстрирует простые, ответственные и дисциплинированные методы разработки программного обеспечения. Оно иллюстрирует множество практических моментов, которые можно применять независимо от используемых методов и процессов. Эти методы в основном изложены с точки зрения программиста, начиная от того, как избежать типичных ошибок кодирования и проектирования, и заканчивая вопросами коммуникации и командной работы.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Хотя эта книга, безусловно, не является всеобъемлющим источником информации о запуске программного
проекта, она представляет собой хороший источник практических советов, которые, вероятно, будут полезны
любому программисту или дизайнеру.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

4. Сравнение гибких методов

Цель данной главы — сравнить методы гибкой разработки программного обеспечения, представленные в главе 3. Глава организована следующим образом. Сначала кратко описывается метод анализа, лежащий в основе сравнения. Затем следует сравнение выбранных общих характеристик. Наконец, рассматриваются некоторые аспекты применения метода и определяются потребности в будущих исследованиях.

4.1 Введение

Задача объективного сравнения практически любой методологии с другой сложна, и результат часто основан на субъективном опыте практикующего специалиста и интуиции авторов (Сонг и Остервейл, 1991). Существуют два альтернативных подхода: неформальное и квазиформальное сравнение (Сонг и Остервейл, 1992). Квазиформальное сравнение пытается преодолеть субъективные ограничения неформального метода сравнения. Согласно Солу (1983), квазиформальные сравнения можно проводить пятью различными способами:

1. Опишите идеализированный метод и сравните с ним другие методы.
2. Выделите набор важных характеристик индуктивным методом из нескольких методов и сравните каждый метод с ним.
3. Сформулируйте априорные гипотезы о требованиях метода и выведите структуру из эмпирических данных по нескольким методам.
4. Дайте определение метаязыку как средству коммуникации и системе отсчета.
против которого вы описываете множество методов.
5. Используйте подход, основанный на ситуативных обстоятельствах, и попытайтесь связать особенности каждого метода с конкретными проблемами.

Сонг и Остервейл (1992) утверждают, что второй и четвертый типы подходов ближе к классическому научному методу, используемому для сравнения методов. Фактически, четвертый подход, то есть детализация метаязыка как средства коммуникации,

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

средство, было операционализировано в предыдущей главе, в которой каждый метод был изложен с использованием одной и той же общей типологии, т.е. процесса, ролей и обязанностей, практик, принятия и опыта, сферы использования и текущих исследований.

Сравнение часто подразумевает оценку одного метода по сравнению с другим. Однако это не является нашей целью. При использовании второго подхода некоторые важные характеристики метода и его применения выбираются в качестве точек зрения, через которые анализируются методы. Таким образом, наша цель — выявить различия и сходства между различными методами гибкой разработки программного обеспечения. Недавно аналогичный подход был использован в исследовании методов анализа архитектуры программного обеспечения (Dobrica and Niemelä, 2002).

4.2 Общие характеристики

В связи с аналитическими элементами, используемыми в главе 3, общие характеристики относятся к текущим исследованиям и области применения. Кроме того, мы будем использовать термины «ключевые моменты» и «особенности». Ключевые моменты описывают основные аспекты или решения методов. Специальные характеристики описывают один или несколько аспектов методов, отличающих их от других.

DSDM и Scrum были представлены в начале и середине 1990-х годов соответственно. Однако при определении статуса метода Scrum всё ещё находится в стадии «становления», поскольку исследований, упоминающих его применение, пока мало. Среди других широко распространённых методов — FDD, Crystal, PP и ASD. Однако об их реальном применении известно меньше. Таким образом, их также можно считать находящимися в стадии «становления». AM был предложен менее года назад, поэтому его статус — «зарождающийся». XP, RUP, OSS и DSDM, с другой стороны, — это хорошо документированные методы и подходы, по которым имеется ряд литературы и отчётов об опыте применения. Более того, каждый метод породил собственные активные исследовательские и пользовательские сообщества. Таким образом, их можно отнести к категории «активных».

Ни один из методов нельзя отнести к категории «угасающих», хотя, например, терминология DSDM, такая как «прототипирование», считается устаревшей. В практическом применении прототипирование можно было бы отнести к гибкому определению «рабочего процесса».

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Код (Highsmith, 2002). Прототипы производительности и производительности затем можно сопоставить с пиком XP, представляющим собой фрагмент тестового кода для конкретного системного аспекта, который можно отбросить. Состояние гибких методов разработки программного обеспечения представлено в таблице 7.

Таблица 7. Статус метода Agile SW-разработки

Статус (8/02)	Описание	Метод
Зарождающийся	Метод был доступен менее одного Год. Не существует достоверных исследований, нет сообщений о подобном опыте.	являюсь
Наращивание	Метод или подход широко признан, появляются отчеты об опыте, формируется активное сообщество, исследования, касающиеся метода, идентифицируются.	FDD, Кристалл, Scrum, PP, ASD
Активный	Метод подробно описан в нескольких местах, отчеты об опыте, активные исследования, активное сообщество.	xP, RUP, OSS, ДСДМ ¹⁵

Как указано в главе 2, все гибкие методы (за исключением RUP и OSS) основаны на хаордической перспективе, все они разделяют схожий набор ценностей и принципов сотрудничества и ценят наличие едва достаточной методологии (Highsmith, 2002). Однако каждый метод подходит к проблемам, с которыми сталкивается программная инженерия, с разных сторон. Таблица 8 суммирует каждый метод, используя три выбранных аспекта: ключевые моменты, особенности и выявленные недостатки. Как указано, ключевые моменты подробно описывают основные аспекты или решения методов, а особые особенности описывают один или несколько аспектов методов, которые отличают их от других. Наконец, выявленные недостатки относятся к некоторым аспектам метода, которые явно отсутствуют, или к другим аспектам, которые были задокументированы в литературе.

¹⁵ DSDM разрабатывается и используется преимущественно членами Консорциума DSDM. Однако методическое руководство находится в открытом доступе.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
 Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
 Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Таблица 8. Общие характеристики методов гибких разработок программного обеспечения.

Метод имя	Ключевые моменты	Специальные возможности	Идентифицировано недостатки
Адаптивная культура РАС, сотрудничество,	целестремленность	Организации рассматриваются как адаптивные системы.	РАС больше касается концепции и культура, чем программное обеспечение упражняться.
	итеративный на основе компонентов	Создание нового порядка из сети	
	разработка	взаимосвязаны отдельных лиц.	
АМ Применение гибких принципов к моделированию:	гибкая культура, организация	Гибкое мышление применимо и к моделированию.	Это хорошая дополнительная философия для профессионалов моделирования.
	работы для поддержки		Однако это работает только в других методы.
	коммуникации, простота.		
Семейство методов Crystal.	У каждого из них одинаковая	Принципы разработки метода.	Пока рано делать оценки: существуют только два из четырех предложенных методов.
	основные ценности и принципы.	Умение выбирать наиболее подходящий метод	
	Методы, роли, инструменты и стандарты	в зависимости от размера и критичности проекта	
	отличаться.		
Применение DSDM к средствам управления RAD, использование временных рамок,	наделение полномочиями команд DSDM, активный консорциум для управления	Первое по-настоящему гибкое программное обеспечение метод разработки, использование прототипирования, несколько ролей пользователей: «посол»,	В то время как метод доступно, только членам консорциума иметь доступ к белому документы, касающиеся фактического использования

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
 Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
 Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/publications/2002/P478.pdf>.

Метод имя	Ключевые моменты	Специальные возможности	Идентифицировано недостатки
	метод разработка.	«визионер» и «советник».	метод.
XP	Ориентированный на клиента разработка, небольшие команды, ежедневные сборки	Рефакторинг — постоянная переработка системы с целью повышения ее производительности и скорости реагирования на изменения.	В то время как отдельные практики подходят для многих ситуаций, общий вид и Меньше внимания уделяется методам управления.
FDD. Пятиэтапный процесс,	объектно- ориентированная разработка компонентов (т.е. функций). Очень короткие часов до 2 недель.	Простота метода, проектирование и реализация системы с помощью функций, моделирование объектов. итерации: от	FDD фокусируется только на проектировании и реализации. Нужны другие вспомогательные подходы.
ОСС	Основанный на волонтерах, распределенный развития, часто проблемная область является более вызов, чем коммерческий начинание.	Практика лицензирования; исходный код доступен всем бесплатно вечеринки.	ОСС — это не метод сам; способность трансформировать ОСС принципы сообщества к коммерческому программному обеспечению разработка.
ПП	Акцент на прагматизме, теория программирования имеет меньшее значение, высокий уровень автоматизации во всех аспектах	Бетон и эмпирически проверенные советы и подсказки, т.е. прагматичный подход к программному обеспечению	ПП фокусируется на важные индивидуальные практики. Однако это не метод через которые система может быть

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Метод имя	Ключевые моменты	Специальные возможности	Идентифицировано недостатки
	программирование.	разработка.	развитый.
Полная модель разработки ПО	RUP, включая поддержку инструментов. Роль, управляемая деятельностью. назначение.	Бизнес-моделирование, поддержка семейства инструментов.	RUP не имеет ограничения в Область применения. Отсутствует описание того, как конкретно адаптироваться к меняющимся потребностям.
Scrum Независимые, небольшие,	самоорганизующиеся команды разработчиков, 30-дневные циклы релиза.	Обеспечить смену парадигмы с «определено и «повторяемого» к «новому взгляду на разработку продукта Scrum».	В то время как детали Scrum в частности, не описывается, как управлять 30- дневным циклом выпуска, интеграционными и приемочными тестами.

В таблице 8 показаны различия в ключевых моментах, на которых акцентируются исследуемые методы. ASD является наиболее абстрактным методом с точки зрения разработки программного обеспечения. Несмотря на всю привлекательность, его ключевая цель — «создание возникающего порядка из сети взаимосвязанных индивидов» — может быть трудной для понимания. Это также его главный недостаток, поскольку практики могут испытывать трудности с переводом «новых» концепций метода на свой пользовательский интерфейс. Гибкое моделирование (AM), XP и прагматическое программирование — все они представляют собой практико-ориентированные точки зрения. Все они содержат ряд эмпирически подтвержденных практик, которые практики считают полезными. Как таковые, они очень ценны. Семейство методологий Crystal — единственное, где явно предлагаются принципы разработки методов, позволяющие адаптировать их в зависимости от размера и критичности проекта. Это важный аспект, поскольку масштабируемость метода является одной из основных тем, которые необходимо рассмотреть сообществу Agile. Попытки в этом направлении также можно отметить (см. обсуждение, например, в eWorkshop 2002).

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Метод DSDM отличается от других методов использованием прототипирования. DSDM также выделяет некоторые роли, которые другие не рассматривали. К таким ролям пользователей (или связанных с заказчиком) относятся посол, визионер и консультант. Эти роли представляют различные точки зрения заказчиков. Недостатком DSDM является требование членства в консорциуме DSDM для получения доступа к техническим документам, описывающим различные аспекты метода. FDD не пытается предоставить комплексное решение для разработки программного обеспечения, а фокусируется на простом пятиэтапном подходе, основанном на выявлении, проектировании и реализации функций. FDD предполагает, что определённая работа по проекту уже выполнена. Следовательно, он не охватывает ранние фазы проекта (более подробную информацию о поддержке жизненного цикла см. в следующем подразделе). Scrum — это подход к управлению проектами, основанный на самоорганизующихся независимых командах разработчиков, реализующих программный проект в рамках 30-дневных циклов, называемых спринтами. Методическое руководство не очень ясное, например, интеграционные и приемочные тесты не описаны.

Подобно ASD, методология открытого программного обеспечения (OSS) представляет собой скорее философию разработки, чем метод как таковой. Тем не менее, в литературе описано множество успешных проектов. Особенностью OSS является практика лицензирования, которая требует, чтобы исходный код был доступен всем сторонам. Исходный код может быть прочитан, изменен, скомпилирован и затем распространен бесплатно. Наконец, RUP не считается особенно гибким методом разработки программного обеспечения, но может им быть. Он выделяется среди других тем, что представляет собой полноценный набор инструментов для разработки, поддерживаемый различными коммерческими инструментами. Этому зачастую не хватает другим методам. RUP также расширяет метод, позволяя включать бизнес-приложения. Методы моделирования, аналогичные DSDM, обеспечивают поддержку и на ранних этапах разработки программного обеспечения.

4.3 Усыновление

В связи с аналитическими элементами, использованными в главе 3, под внедрением здесь подразумеваются процесс, роли и обязанности, а также внедрение и опыт. Кроме того, сравнение будет расширено и включит в себя точку зрения управления проектами, которая проливает свет на понимание того, как метод способствует управленческой перспективе.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Программные продукты приносят бизнесу выгоду только при их использовании. Аналогично, преимущества, связанные с методами гибкой разработки программного обеспечения, достигаются только при их использовании в производственном процессе. Внедрение новой технологической технологии должно быть относительно простым, поскольку организации не могут позволить себе замедлять или останавливать производство для реорганизации или освоения новых методов ведения бизнеса (Krueger, 2002). Внедрение и опыт применения каждого метода кратко рассматривались в главе 3. Очевидно, что отчетов об опыте внедрения немного. Научные исследования на данном этапе найти ещё сложнее.

Нандхакумар и Ависон (1999, стр. 188) обнаружили, что в настоящее время традиционные методы разработки программного обеспечения используются «как необходимая фикция для представления образа контроля или обеспечения символического статуса». Они предполагают, что «требуется альтернативные подходы, учитывающие особый характер работы в таких условиях». Особый характер работы, о котором они говорили, — это турбулентная бизнес-среда или «экономика изменений», как её называют сторонники гибких методологий (например, Хайсмит, 2002). В такой среде ожидается, что изменения в проекте появятся даже на очень поздней стадии. Нандхакумар и Ависон (1999) обнаружили, что методы работы разработчиков лучше всего характеризуются импровизацией, когда члены команды защищают свою профессиональную автономию, а социальный контроль оказывает на их практику большее влияние, чем предлагаемые методологии. Это, опять же, проблемы, которые все методы гибкой разработки программного обеспечения решают в явном виде. Фактически, все рассмотренные гибкие методы можно охарактеризовать как делающие акцент на следующих аспектах: 1) предоставление чего-то полезного, 2) опора на людей, 3) поощрение сотрудничества, 4) содействие техническому совершенству, 5) выполнение максимально простых действий и 6) постоянная адаптация (Highsmith 2002).

Нандхакумар и Ависон (1999) утверждали, что современная разработка ПО существенно отличается от общепринятого методологического подхода (см. также главу 2 для обсуждения). Таблица 9 дополняет их выводы с точки зрения гибкого подхода. Кроме того, в таблице 9 показано, что гибкие подходы к разработке ПО во многом воплощают современную практику разработки ПО. Таким образом, это отчасти объясняет, почему, например, экстремальное программирование привлекло внимание в последние годы. Оно явно рассматривает разработку ПО с точки зрения разработчиков.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
 Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
 Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Таблица 9. Сравнение взглядов на разработку программного обеспечения (Нандакумар и Ависон, 1999), дополненных точкой зрения гибких методов.

	Методологический взгляд	Программное обеспечение Развитие на практике	Представленная точка зрения гибкими методами
Деятельность	Дискретные задачи	Взаимосвязанные личные проекты	Взаимосвязанные личные проекты
	Продолжительность предсказуемый	Завершение непредсказуемо	Завершение следующего релиза предсказуемый
	Повторяемый	Зависит от контекста Часто зависит от контекста	
Процесс Производительность	Надежный	Зависит от контекста условия	Производительность связана с небольшой релиз, таким образом надежный
	Определенные взаимодействия	По своей сути интерактивный	Определенные взаимодействия продвижение изначально интерактивной природы
	Отдельные задачи в последовательность	Многие задачи переплетены	Отдельные задачи часто не являются обязательными из-за к их контексту зависимая природа

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

	Методологический взгляд	Программное обеспечение Развитие на практике	Представленная точка зрения гибкими методами
Усилия разработчиков	Посвящено SW проекты	Общее для всех деятельность (например, проектная, непроежная, личная, рутинная)	Разработчики оценивают работу требуемые усилия
	Недифференцированный	Специфические для отдельных лиц	Специфические для отдельных лиц
	Полностью доступно	Полностью использовано	Полностью использовано
Контроль работа	Регулярность	Оппортунизм, импровизация и прерывание	Существуют только взаимно согласованные меры контроля ¹⁶ , уважение к работа других
	Контрольные точки прогресса, планирование и контроль управления	Индивидуальный предпочтения и взаимные переговоры	Индивидуальный предпочтения и взаимные переговоры

Хотя гибкие подходы соответствуют современной практике разработки программного обеспечения, не все они подходят для всех этапов жизненного цикла разработки программного обеспечения.

На рисунке 14 показано, какие этапы разработки программного обеспечения поддерживаются различными гибкими методологиями. Каждый метод разделён на три блока. Первый блок показывает, обеспечивает ли метод поддержку управления проектами. Второй блок определяет, описан ли в нём процесс, который метод предлагает использовать. Третий блок показывает,

¹⁶ Это не относится к некоторым гибким методам. Например, DSDM и Crystal.
 подробно определить используемые элементы управления.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Метод описывает практики, виды деятельности и рабочие продукты, которые могут применяться и использоваться в различных обстоятельствах. Серый цвет в блоке означает, что метод охватывает фазу жизненного цикла, а белый цвет означает, что метод не предоставляет подробной информации ни об одной из трёх оцениваемых областей: управлении проектами, процессе или практиках/видах деятельности/рабочих продуктах.

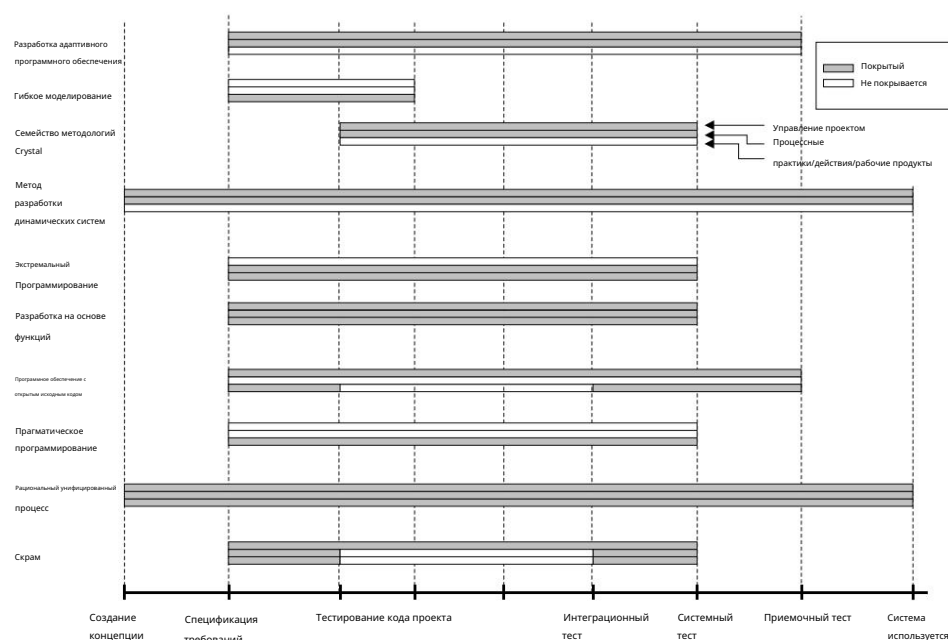


Рисунок 14. Поддержка жизненного цикла разработки программного обеспечения

На рисунке 14 показано, что гибкие методологии разработки программного обеспечения фокусируются на различных аспектах жизненного цикла разработки программного обеспечения. Более того, некоторые из них больше ориентированы на практику (экстремальное программирование, прагматичное программирование, гибкое моделирование), в то время как другие сосредоточены на управлении программными проектами (Scrum). Тем не менее, существуют подходы, обеспечивающие полный охват жизненного цикла разработки (DSDM, RUP), при этом большинство из них подходят, начиная с этапа спецификации требований (например, FDD). Таким образом, в этом отношении существует четкое различие между различными гибкими методами разработки программного обеспечения. В то время как DSDM и RUP не требуют дополнительных подходов для поддержки разработки программного обеспечения, другие требуют их в той или иной степени. Однако DSDM доступен только для членов

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Консорциум DSDM и RUP, таким образом, представляют собой коммерчески продаваемую среду разработки.

Учитывая внедрение этих методов, размер команды разработчиков в настоящее время является одним из важнейших определяющих факторов. XP, Scrum, AM и PP ориентированы на небольшие команды, предпочтительно менее 10 инженеров-программистов. Crystal, FDD, RUP, OSS, ASD и DSDM заявляют о возможности масштабирования до 100 разработчиков. Однако сторонники Agile сходятся во мнении, что с ростом команды разработчиков, объём, например, документации, вероятно (и должен) увеличиться, что сделает проект «менее гибким». Когда группа разработчиков превышает 20 инженеров-программистов, сторонники Agile уделяют особое внимание эффективному решению коммуникационных задач (Ambler, 2002c). Каждый метод содержит ряд рекомендаций по организации каналов коммуникации внутри небольшой группы инженеров.

Гибкий подход представляется особенно подходящим в ситуации, когда будущие требования неизвестны (например, Ambler 2002c). Похоже, что если будущие требования (требования к производительности, обработка исключений, функциональность и т. д.) известны и могут быть определены, гибкие подходы сами по себе не добавляют ценности проекту разработки. Хайсмит (2002) предпринял попытку определить, какая рыночная фаза наиболее подходит для гибких методов. Он предполагает, что гибкие подходы особенно подходят для фаз «боулинг» и «торнадо» (см. характеристики различных рыночных фаз в Moore 1995). В качестве примера, в боулинге, утверждает Хайсмит, «высокий уровень кастомизации и интенсивное взаимодействие с клиентами могут быть улучшены с помощью гибкого подхода, ориентированного на сотрудничество». Хайсмит также считает важным соответствие типа методологии культуре организации. Организации, которые ценят компетентность и интенсивное сотрудничество, больше подходят для гибких подходов, чем те, которые полагаются на контроль или развитие. Однако эмпирических подтверждений этим утверждениям не предоставлено.

Стоимость и сложность внедрения сложно оценить, поскольку опубликовано лишь очень ограниченное количество исследований, посвящённых этим вопросам. Однако мы считаем, что если смена парадигмы между традиционным подходом к разработке программного обеспечения и гибкой разработкой слишком существенна, а поддерживающие модели и методы внедрения отсутствуют, организации и разработчики, скорее всего, не будут заинтересованы во внедрении гибких методов.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Повседневное использование. Этот сдвиг парадигмы подразумевает придание большего значения другим вопросам, выходящим за рамки традиционного внимания, таким как планирование, подготовка к будущему, документирование, переговоры по контрактам, а также процессы и инструменты. Организации и люди вряд ли смогут кардинально изменить свою практику разработки программного обеспечения.

Большинство изученных методов предполагают предоставление заказчику и команде разработчиков возможности принимать важные решения, касающиеся процесса разработки. Таким образом, внедрение гибких практик разработки ПО также требует изменения культуры, особенно в среднем и высшем руководстве. Многие традиционные механизмы контроля, такие как периодическая отчетность, смещаются в сторону создания работающего кода. С одной стороны, это требует изменения принципов функционирования организации и, что еще важнее, с другой стороны, повышения доверия к способностям и компетентности команды разработчиков. С другой стороны, существует риск, связанный с изменением подхода к разработке.

Все гибкие подходы предполагают разработку функций программного обеспечения небольшими инкрементными циклами, продолжительностью всего от 2 до 6 недель. Таким образом, риск сводится к минимуму, поскольку, например, отставание от графика станет заметным уже через несколько недель.

Известные сторонники гибких методов, такие как Хайсмит и Швабер, считают, что лучшие результаты внедрения достигаются, когда «проект находится «прижатым к стенке» (eWorkshop 2002; Highsmith 2002), что обеспечивает внимание и поддержку руководства. Хотя подобные примеры из практики полезны, необходимы эмпирически подтвержденные или чисто экспериментальные исследования, чтобы определить, когда, как и в каких ситуациях гибкие методы лучше всего использовать и применять.

Внедрение новых технологий – хорошо изученная область исследований (более подробную информацию и соответствующие ссылки см., например, Sultan and Chan, 2000). Однако маловероятно, что технологии гибкой разработки программного обеспечения настолько радикально отличаются от других программных технологий, что это могло бы снизить ценность исследований в области распространения инноваций. Например, Агарвал и Прасад (2000) показывают, что представления о новых технологиях и их последующее внедрение тесно связаны. Абрахамссон (2002) приводит аналогичные аргументы. Другими важными факторами, влияющими на внедрение новых технологий, являются стаж работы в организации, предыдущие технические знания, опыт обучения и воспринимаемая неуверенность в завтрашнем дне. Эти результаты, среди прочего, следует использовать при разработке моделей внедрения.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

призван предоставить руководство в процессе внедрения методов гибкой разработки
программного обеспечения.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

5. Выводы

Исследования показали, что традиционные методологии разработки программного обеспечения, основанные на планировании, на практике не применяются. Утверждается, что традиционные методологии слишком механистичны для детального использования. В результате разработчики промышленного программного обеспечения стали скептически относиться к «новым» решениям, которые трудно понять и, следовательно, остаются неиспользуемыми. Методы гибкой разработки программного обеспечения, «официально» начавшиеся с публикации Agile-манифеста, пытаются произвести сдвиг парадигмы в области программной инженерии. Agile-методы претендуют на то, чтобы уделять больше внимания людям, взаимодействию, работающему программному обеспечению, сотрудничеству с клиентами и изменениям, а не процессам, инструментам, контрактам и планам. Был представлен ряд новых многообещающих методологий, претендующих на соответствие этим принципам Agile. Тем не менее, систематического обзора гибких методов не проводилось.

Эта публикация преследовала три цели. Во-первых, она обобщила существующую литературу о том, что на самом деле означает термин «agile», задав вопрос: «Что делает метод разработки гибким?». Вывод заключался в том, что это относится к разработке программного обеспечения.

- инкрементный (небольшие выпуски программного обеспечения с быстрыми циклами),
- кооперативный (заказчик и разработчики постоянно работают вместе, обеспечивая тесную связь),
- простота (сам метод легко освоить и модифицировать, ну задокументировано), и
- адаптивный (умеющий вносить изменения в последний момент).

Во-вторых, на основе этого определения каждый метод был описан с точки зрения процесса, ролей и обязанностей, практик, внедрения и опыта, а также текущих исследований. В-третьих, это позволило выбрать критерии для сравнения методов и выявить их различия и сходства. Было обнаружено, что, хотя гибкие методы имеют много общего, некоторые из них более узконаправлены, чем другие. Например, они в разной степени поддерживают различные этапы разработки программного продукта. Были также обнаружены различия в

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Уровень конкретности, на котором методы привязаны к реальной практике разработки. Например, ASD преимущественно фокусируется на принципах, определяющих разработку программного обеспечения, тогда как XP делает акцент на практиках разработки. Эти и другие вопросы обсуждались в главе 4.

Однако это лишь отправная точка для нашего исследования. Отдельные данные свидетельствуют об эффективности гибких методов и их пригодности для многих ситуаций и условий. Однако в настоящее время существует очень мало эмпирически подтвержденных исследований, подтверждающих эти утверждения. Существующие данные в основном представляют собой истории успеха практикующих специалистов. Хотя они предоставляют ценную информацию о практическом применении, эмпирические исследования крайне необходимы для оценки эффективности и возможностей использования гибких методов разработки программного обеспечения. Более того, частый выпуск новых гибких методов скорее внесёт путаницу, чем ясность. Каждый метод использует свой собственный словарь и терминологию, и мало что делается для интеграции множества высказанных точек зрения. Сейчас крайне необходимы (в большей степени, чем новые модели) модели принятия или отбора, которые могли бы использовать практикующие специалисты. Цель такого рода исследований — дать возможность специалистам по разработке программного обеспечения, проектам и организациям выбирать и применять правильный метод в нужное время. Мы считаем, что эмпирические исследования или эксперименты предоставляют плодотворную платформу для такого рода разработок.

В заключение следует отметить, что гибкое мышление — это подход к разработке программного обеспечения, ориентированный на людей. Стратегии, ориентированные на людей, рассматриваются как важный источник конкурентного преимущества, поскольку, в отличие от технологий, затрат или разработки новых продуктов, эти стратегии, ориентированные на людей, трудно имитировать (Pfeffer, 1998; Miller and Lee, 2001). Однако это не новое открытие. Летний выпуск журнала *American Programmer* 1990 года (журнал Ed Yourdon's *Software Journal*, том 3, № 7-8) был посвящен исключительно теме «Peopleware». Редактор комментирует этот специальный выпуск следующим образом: «Все знают, что лучший способ повысить производительность и качество программного обеспечения — это сосредоточиться на людях». Таким образом, идеи, которые несут в себе гибкие методы, не новы, и сторонники гибких методологий этого не утверждают. Мы, однако, считаем, что гибкие методы разработки программного обеспечения предоставляют новый подход к решению задач программной инженерии, утверждая при этом, что эти методы ни в коем случае не являются исчерпывающими и не способны решить все проблемы. Большая часть программного обеспечения в крупных организациях не производится совместно

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Команды менее чем из десяти инженеров. Если, например, не будут решены проблемы масштабируемости,
гибкое мышление не получит того серьёзного внимания, которого оно заслуживает.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Ссылки

Абрахамссон, П. (2002). «Сети обязательств в улучшении процесса разработки программного обеспечения». *Анналы Программная инженерия*, в печати.

Агарвал, Р. и Дж. Прасад (2000). «Полевое исследование внедрения инноваций в процесс разработки программного обеспечения профессионалами в области информационных систем». *Труды IEEE по инженерному менеджменту* 47(3): 295-308.

Александр, К. (1979). *Вневременной способ строительства*. Нью-Йорк: Oxford University Press.

Эмблер, С. (2002a). *Гибкое моделирование: эффективные методы экстремального программирования и унифицированного процесса*. Нью-Йорк: John Wiley & Sons, Inc., Нью-Йорк.

Эмблер, С. (2002b). «Уроки гибкости, извлеченные из интернет-разработки». *IEEE Software* Март / Апрель: 66 - 73.

Эмблер, С.В. (2002c). «В борьбе». *Разработка программного обеспечения* 10.

Андерсон, А., Р. Битти, К. Бек, Д. Брайант, М. ДеАрмент, М. Фаулер, М. Фрончак, Р. Гарзанити, Д. Гор, Б. Хакер, К. Хэндриксон, Р. Джеффрис, Д. Джоппи, Д. Ким, П. Ковальски, Д. Мюллер, Т. Мурашки, Р. Наттер, А. Пантеа и Д. Томас (1998). «Chrysler идёт к крайностям». *Пример из практики. Распределённые вычисления (октябрь)*: 24-28.

Баскервиль, Р. (1998). *Проектирование безопасности информационных систем*. Чичестер, Великобритания, John Wiley.

Баскервиль, Р., Дж. Трэвис и Д.П. Труэкс (1992). *Системы без метода: влияние новых технологий на проекты разработки информационных систем. Труды о влиянии компьютерных технологий на разработку информационных систем*. К.Е. Кендалл, К. Люйтинен и Дж.И. ДеГросс. Амстердам, Научные публикации Elsevier: 241-260.

Байер, С. и Дж. Хайсмит (1994). «RADical разработка программного обеспечения». *American Programmer* 7(6): 35-42.

Бек, К. (1999a). «Принятие изменений с помощью экстремального программирования». *IEEE Computer* 32(10): 70-77.

Бек, К. (1999b). *Экстремальное программирование*. Реддинг, Массачусетс, Эддисон-Уэсли.

Бек, К. (2000). *Экстремальное программирование: примите изменения*.

Бек, К., М. Бидл, А. Беннекум ван, А. Кокберн, В. Каннингем, М. Фаулер, Дж. Греннинг, Дж. Хайсмит, А. Хант, Р. Джеффрис, Дж. Керн, Б. Марик, Р. Мартин, С. Меллор, К. Швабер, Дж. Сазерленд и Д. Томас (2001). «Манифест гибкой разработки программного обеспечения». 2002 (22.3.2002) <http://AgileManifesto.org>.

Бергквист, М. и Дж. Юнгберг (2001). «Сила даров: организация социальных отношений в сообществах разработчиков ПО с открытым исходным кодом». *Information Systems Journal* 11(4): 305-320.

Бём, Б. (2002). «Готовьтесь к гибким методам с осторожностью». *Computer* 35(1): 64-69.

Кoad, П., Э. Лефевр и Дж. Де Лука (2000). *Моделирование Java в цвете с использованием UML: компоненты и процессы предприятия*, Prentice Hall.

Кокберн, А. (1998). *Выживание объектно-ориентированных проектов: руководство для менеджеров*, Эддисон Уэсли Лонгман.

Кокберн, А. (2000). *Написание эффективных вариантов использования*, The Crystal Collection for Software Профессионалы, Эддисон-Уэсли Профессионал.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Кокберн, А. (2002а). Гибкая разработка программного обеспечения. Бостон: Эддисон-Уэсли.

Кокберн, А. (2002b). «Гибкая разработка программного обеспечения присоединяется к толпе „потенциальных“». Cutter IT Journal 15(1): 6-12.

Коплиен, Дж. О. (1998). Язык шаблонов процесса генеративного развития. Справочник по шаблону. Л. Райзинг. Нью-Йорк: Издательство Кембриджского университета: 243-300.

Койн, Р. (1995). Проектирование информационных технологий в эпоху постмодерна. Кембридж, Массачусетс.

Кроустон, К. и Б. Скоцци (2002). «Проекты разработки программного обеспечения с открытым исходным кодом как виртуальные организации: Повышение компетентности в разработке программного обеспечения». Труды IEEE - Программное обеспечение 149(1): 3-17.

Каннингем, У. (1996). Эпизоды: язык шаблонов конкурентного развития. Языки шаблонов проектирования программ. 2. Дж. Влассидес. Нью-Йорк: Эддисон-Уэсли.

ДеМарко, Т. и Т. Листер (1999). Peopleware. Нью-Йорк, Dorset House.

Демпси, Б., Д. Вайс, П. Джонс и Дж. Гринберг (2002). «Кто есть и программное обеспечение с открытым исходным кодом Разработчик». SACM 45(2): 67 - 72.

Диллон, Г. (1997). Управление безопасностью информационных систем. Лондон, Великобритания, MacMillan Press.

Добрица, Л. и Э. Ниемея (2002). «Обзор методов анализа архитектуры программного обеспечения». IEEE Труды по программной инженерии 28(7): 638-653.

DSDMConsortium (1997). Метод разработки динамических систем, версия 3. Эшфорд, Энг., DSDMConsortium.

Электронный семинар (2002). «Краткое содержание второго электронного семинара по гибким методам». 2002 (8 августа) <http://fc-md.umd.edu/projects/Agile/SecondeWorkshop/summary2ndeWorksh.htm>.

Фаваро, Дж. (2002). «Управление требованиями для повышения ценности бизнеса». IEEE Software, март/апрель: 15 - 17.

Феллер, Дж. и Б. Фицджеральд (2000). Анализ структуры программного обеспечения с открытым исходным кодом. Парадигма развития. 21-я ежегодная международная конференция по информационным системам, Брисбен, Австралия.

Галливан, М. (2001). «Нахождение баланса между доверием и контролем в виртуальной организации: контент-анализ примеров программного обеспечения с открытым исходным кодом». Information Systems Journal 11(4): 273-276.

Гош, Р. и В. Пракаш (2000). «Обзор свободного программного обеспечения Orbiteen». 2002(03052002) orbiteen.org/ofss/01.html.

Гилб, Т. (1988). Принципы управления разработкой программного обеспечения. Уокингем, Великобритания, Эддисон-Уэсли.

Гласс, Р.Л. (1999). «Реалии выгод от программных технологий». Сообщения ACM 42(2): 74-79.

Гласс, Р.Л. (2001). «Agile против традиционного: занимайтесь любовью, а не войной!» Cutter IT Journal 14(12): 12-18.

Греннинг, Дж. (2001). «Внедрение XP в компании с интенсивной производственной деятельностью». IEEE Software 18: 3-9.

Хаунгс, Дж. (2001). «Парное программирование в проекте С3». Компьютер 34(2): 118-119.

Хавриш, С. и Дж. Рупрехт (2000). Методологии света: снова как дежавю. Журнал Cutter IT. 13: 4 - 12.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Хайсмит, Дж. (2002). Экосистемы гибкой разработки программного обеспечения. Бостон, Массачусетс, Pearson Education.

Хайсмит, Дж. и А. Кокберн (2001). «Гибкая разработка программного обеспечения: бизнес инноваций». Computer 34(9): 120-122.

Хайсмит, Дж. А. (2000). Адаптивная разработка программного обеспечения: совместный подход к управлению сложными системами. Нью-Йорк, издательство Dorset House.

Хайтауэр, Р. и Н. Лесецки (2002). Инструменты Java для экстремального программирования. Нью-Йорк, Wiley Компьютерное издательство.

Хамфри, У. С. (1995). Дисциплина в области программной инженерии, Addison Wesley Longman, Inc.

Хант, А., Томас, Д. (2000). «Программист-прагматик», Эддисон Уэсли.

Якобсен, И. (1994). Объектно-ориентированная программная инженерия. Нью-Йорк: Эддисон-Уэсли.

Якобсен, И., М. Кристерсон, П. Йонссон и Г. Овергаард (1994). Объектно-ориентированная программная инженерия: подход, основанный на вариантах использования. Реддинг, Массачусетс, Эддисон-Уэсли.

Джеффрис, Р., А. Андерсон и Х.К. (2001). Экстремальное программирование. Аппер Сэддл Ривер, Нью-Джерси, Эддисон-Уэсли.

Крухтен, П. (1996). «Рациональный процесс развития». Crosstalk 9(7): 11-16.

Крухтен, П. (2000). Рациональный унифицированный процесс: введение, Эддисон-Уэсли.

Крюгер, К. (2002). «Устранение барьера внедрения». IEEE Software 19(4): 29-31.

Лакофф, Г. и Джонсон М. (1998). Философия во плоти. Нью-Йорк, Basic Books.

Лернер, Дж. и Дж. Тироль (2001). «Простая экономика открытого исходного кода». 2002 (20.05.2002)
<http://www.people.hbs.edu/jlerner/publications.html>.

Мартин, Р. К. (1998). Процесс. Объектно-ориентированный анализ и проектирование с приложениями, Addison Wesley: 93-108.

Маурер, Ф. и С. Мартел (2002a). «Экстремальное программирование: Быстрая разработка веб-приложений». приложения». IEEE Internet Computing 6(1): 86-90.

Маурер, Ф. и С. Мартел (2002b). «О производительности гибких методов разработки программного обеспечения: отраслевой пример». (страница проверена 16 августа 2002 г.) <http://sern.ucalgary.ca/~milos/papers/2002/MaurerMartel2002c.pdf>.

Макколи, Р. (2001). «Гибкие методы разработки готовы изменить статус-кво». SIGCSE Бюллетень 33(4): 14 - 15.

Миллер, Д. и Дж. Ли (2001). «Люди создают процесс: обязательства перед сотрудниками, принятие решений создание и исполнение». Журнал управления 27: 163-189.

Миллер, Г.Г. (2001). Характеристики гибких процессов разработки программного обеспечения. 39-я Международная конференция по объектно-ориентированным языкам и системам (TOOLS 39), Санта-Барбара, Калифорния.

Мокус, А., Р. Филдинг и Дж. Хербслеб (2000). Пример программного обеспечения с открытым исходным кодом. Разработка: Apache Server. 22-я Международная конференция по программной инженерии, ICSE 2000, Лимерик, Ирландия.

Мур, Джорджия (1995). Внутри торнадо. Нью-Йорк, HarperBusiness.

Нандхакумар, Дж. и Дж. Ависон (1999). «Фикция методологического развития: полевое исследование развития информационных систем». Информационные технологии и люди 12(2): 176-191.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O., Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

- Наур, П. (1993). «Понимание универсальной машины Тьюринга: индивидуальный стиль описания программ». The Computer Journal 36(4): 351-372.
- О'Рейли, Т. (1999). «Уроки разработки программного обеспечения с открытым исходным кодом». Сообщения ACM, том 42 (№ 4): 32-37.
- Палмер, С.Р. и Фелсинг Дж.М. (2002). Практическое руководство по разработке на основе функций.
- Парнас, Д.Л. и П.К. Клементс (1986). «Рациональный процесс проектирования: как и зачем его имитировать». Труды IEEE по программной инженерии 12(2): 251-257.
- Пфеффер, Дж. (1998). Человеческое уравнение: создание прибыли, ставя людей на первое место. Бостон, Массачусетс, издательство Гарвардской школы бизнеса, копия.
- Пейхёнен, Дж. (2000). Ууси вараллисуоикейс. (На финском языке). Хельсинки, Лакиимеслиитон Кустаннус.
- Райзинг, Л. и Н. С. Янофф (2000). «Процесс разработки программного обеспечения Scrum для небольших команд». Программное обеспечение IEEE 17(4): 26-32.
- Шу, П. (2001). «Восстановление, искупление и экстремальное программирование». IEEE Software 18(6): 34-41.
- Швабер, К. (1995). Процесс разработки Scrum. Семинар OOPSLA'95 по бизнес-объектам Дизайн и реализация, Springer-Verlag.
- Швабер, К. и М. Бидл (2002). Гибкая разработка программного обеспечения с использованием Scrum. Анпер Сэддл Ривер, Нью-Джерси, Прентис-Холл.
- Шарма, С., В. Сугумаран и Б. Раджагопалан (2002). «Структура для создания сообществ разработчиков гибридного программного обеспечения с открытым исходным кодом». Information Systems Journal 12(1): 7-25.
- Смит, Дж. (2001). Сравнение RUP и XP, Технический документ Rational Software: <http://www.rational.com/media/whitepapers/TP167.pdf>.
- Сол, Х. Г. (1983). Анализ особенностей методологий проектирования информационных систем: Методологические соображения. Методологии проектирования информационных систем: анализ характеристик. TW Olle, HG Sol и CJ Tully. Амстердам, Elsevier: 1-8.
- Соммервилл, И. (1996). Программная инженерия. Нью-Йорк: Эддисон-Уэсли.
- Сонг, Х. и Л. Дж. Остервейл (1991). Сравнение методологий проектирования посредством моделирования процессов. 1-я Международная конференция по программным процессам, Лос-Аламитос, Калифорния, IEEE CS Press.
- Сонг, Х. и Л. Дж. Остервейл (1992). «На пути к объективным, систематическим сравнениям методов проектирования». Программное обеспечение IEEE 9(3): 43-53.
- Стэплтон, Дж. (1997). Метод разработки динамических систем – применение метода на практике, Эддисон. Уэсли.
- Суччи, Дж. и М. Маркези (2000). Экстремальное программирование: избранные статьи из XP Конференция XP 2000 года. Конференция XP 2000, Кальяри, Италия, Эддисон-Уэсли.
- Султан, Ф. и Л. Чан (2000). «Внедрение новых технологий: случай объектно-ориентированного программирования». вычисления в компаниях-разработчиках программного обеспечения». Труды IEEE по инженерному менеджменту 47(1): 106-126.
- Такеучи, Х. и И. Нонака (1986). «Игра в разработку нового продукта». Harvard Business Review, январь/февраль: 137-146.

Это авторская версия произведения. Окончательная версия была опубликована в: Abrahamsson, P., Salo, O.,
Ронкайнен, Дж. и Варста, Дж. (2002) Методы гибкой разработки программного обеспечения: обзор и анализ, VTT
Издание 478, Эспоо, Финляндия, 107 стр. Версию правообладателя можно скачать здесь.
<http://www.vtt.fi/inf/pdf/publications/2002/P478.pdf>.

Труэкс, Д.П., Р. Баскервиль и Дж. Трэвис (2000). «Разработка системных методов: отложенное значение методов
разработки систем». Бухгалтерский учет, менеджмент и информационные технологии, 10: 53–79.

Уолл, Д. (2001). «Использование открытого исходного кода для прибыльного стартапа». Computer, декабрь: 158–160.

Варста, Дж. (2001). Контракты в сфере программного обеспечения: анализ развития контрактных процессов и
отношений. Кафедра информационных технологий. Оулу, Университет Оулу, Финляндия: 262.

Вигерс, К.Э. (1998). «Читай по губам: новых моделей нет». IEEE Software 15(5): 10–13.

Уильямс, Л., Р. Р. Кесслер, У. Каннингем и Р. Джеффрис (2000). «Укрепление аргументов в пользу
Парное программирование». IEEE Software 17(4): 19–25.